

Module 1- Introduction to Soft Computing and Neural Network

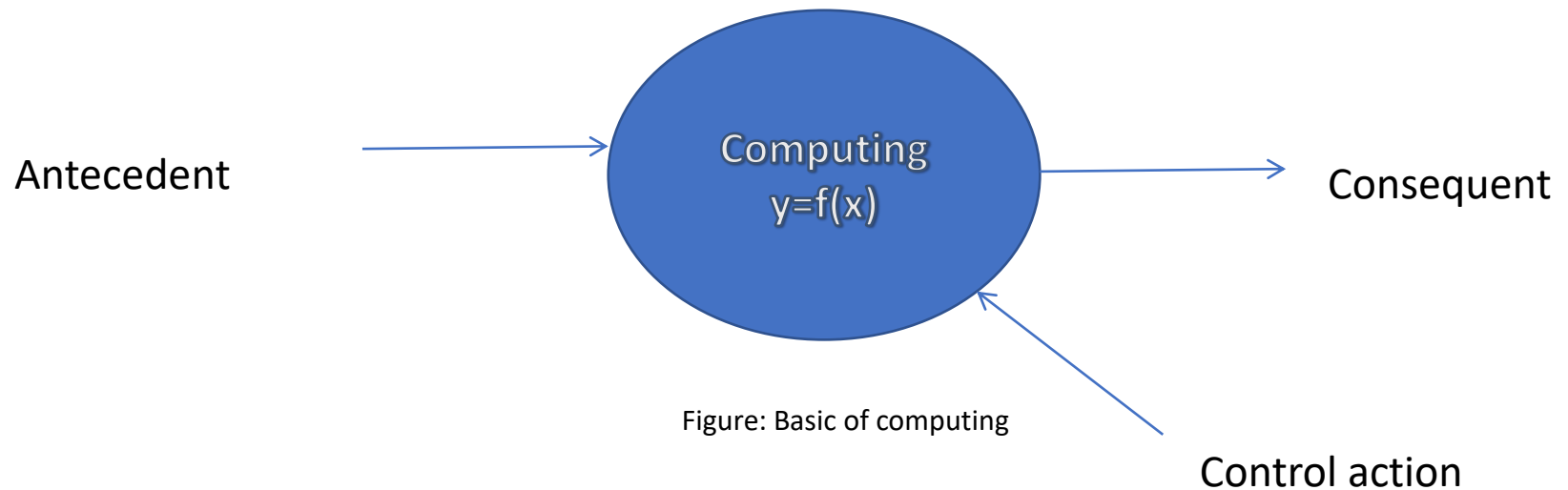
Outline

- Computing System
- Difference - Hard Vs Soft
- Characteristics of Soft Computing
- Applications of Soft Computing
- Biological Neurons
- ANN Terminologies
- Linearly and non-linearly separable classification
- McCulloch Pitts Model
- Perceptron Model

Concept of Computing System

- The **computing system** means that the hardware, the software and the data interact to solve problems.
- The model of all computing systems explains how computing systems work in very simple three words which are “input—process—output”.
- A computer receives an input, processes the information and then performs an output.

Concept of COMPUTATION



$y=f(x)$, f is the mapping function

f is also called a formal method or an **algorithm** to solve a problem

Important characteristic of computing

- Should provide **precise** solution
- Control action should be **unambiguous** and **accurate**.
- Suitable for problem, which is easy to model **mathematically**.

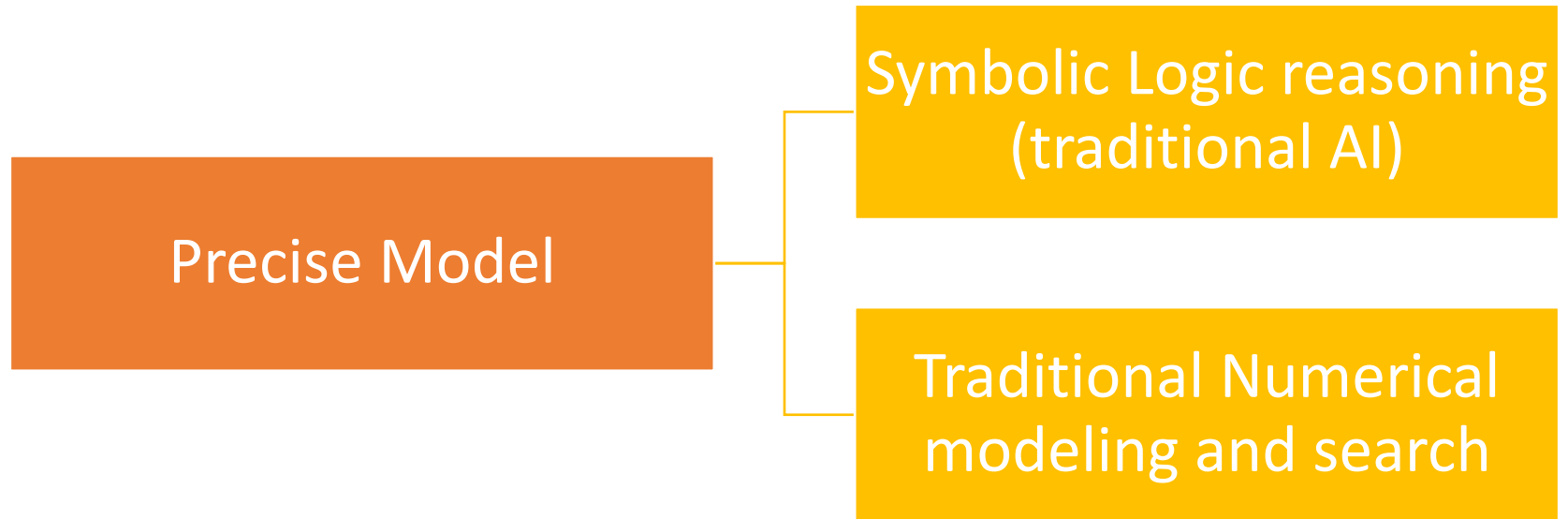
Hard computing

- In 1996, L.A. Zade (LAZ) introduced the term hard computing.
- According to LAZ: we term a computing as Hard computing,
 - **Precise result** is guaranteed
 - Control action are **unambiguous**
 - Control action is **formally defined** (i.e. with mathematical model or algorithm)

Examples of hard computing

- Solving **numerical problems** (e.g., roots of polynomials, integration etc.)
- **Searching and sorting** techniques.
- Solving **computational geometry problems** (e.g., shortest tour in graph, finding closest pair of points given a set of points, etc.)
- Many more...

Hard Computing



Soft Computing

- The term soft computing was proposed by the inventor of fuzzy logic, Lotfi A.Zadeh. He describes it as follows.

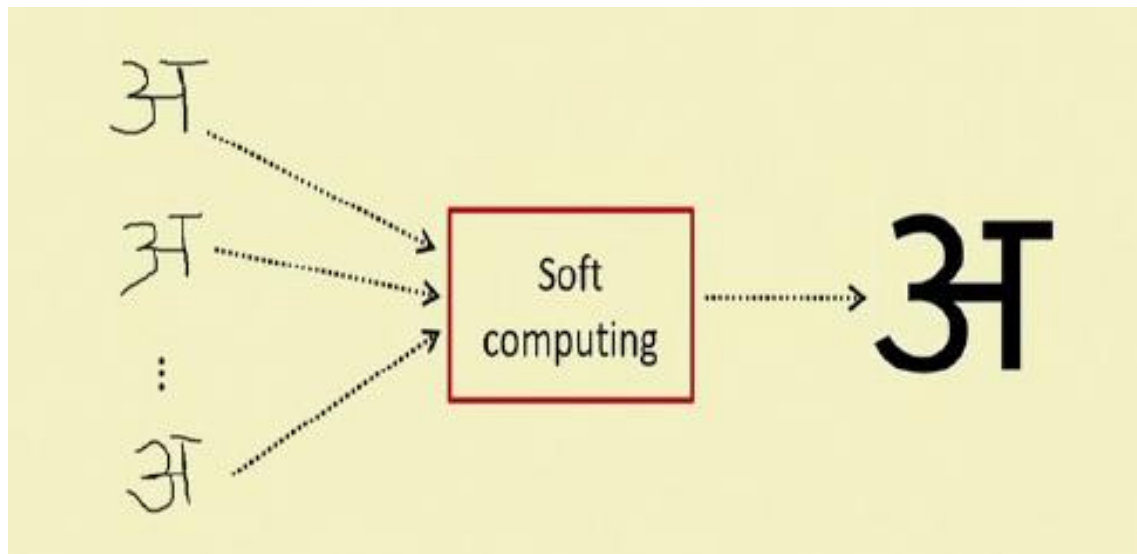
Definition: Soft computing

Soft computing is a collection of methodologies that aim to exploit the **tolerance for imprecision** and **uncertainty** to achieve tractability, **robustness** and **low solution cost**. Its principal constituents are fuzzy logic, neuro-computing and probabilistic reasoning. The role model for soft computing is human mind.

Characteristics of soft computing

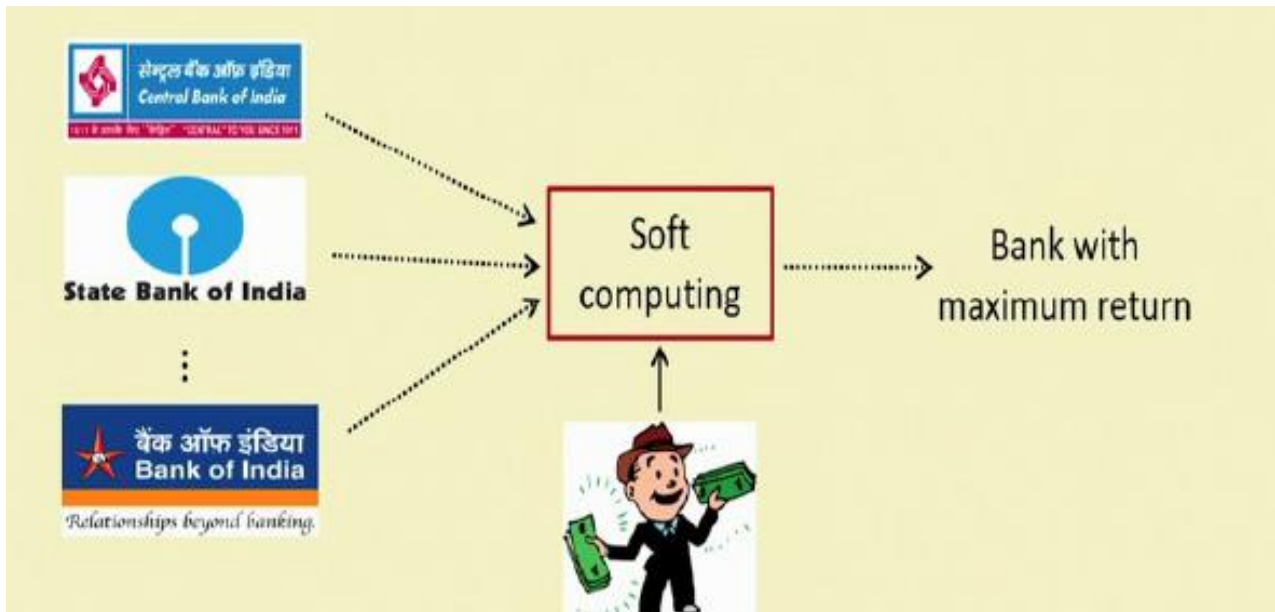
- It **does not require** any mathematical modeling of problem solving.
- It **may not yield** the precise solution.
- Algorithms are **adaptive**(i.e., it can adjust to the change in dynamic environment).
- Use some **biological inspired methodologies** such as genetics, evolution, Ants behaviors, particles swarming, human nervous system etc.)

Examples of soft computing



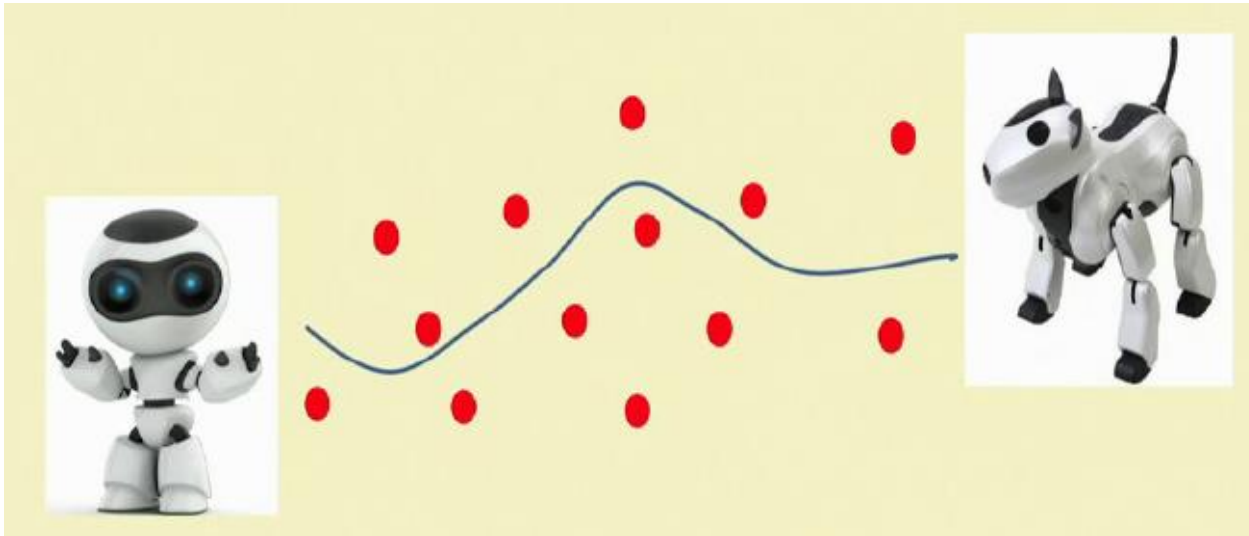
Example: Hand written character recognition
(Artificial Neural networks)

Examples of soft computing



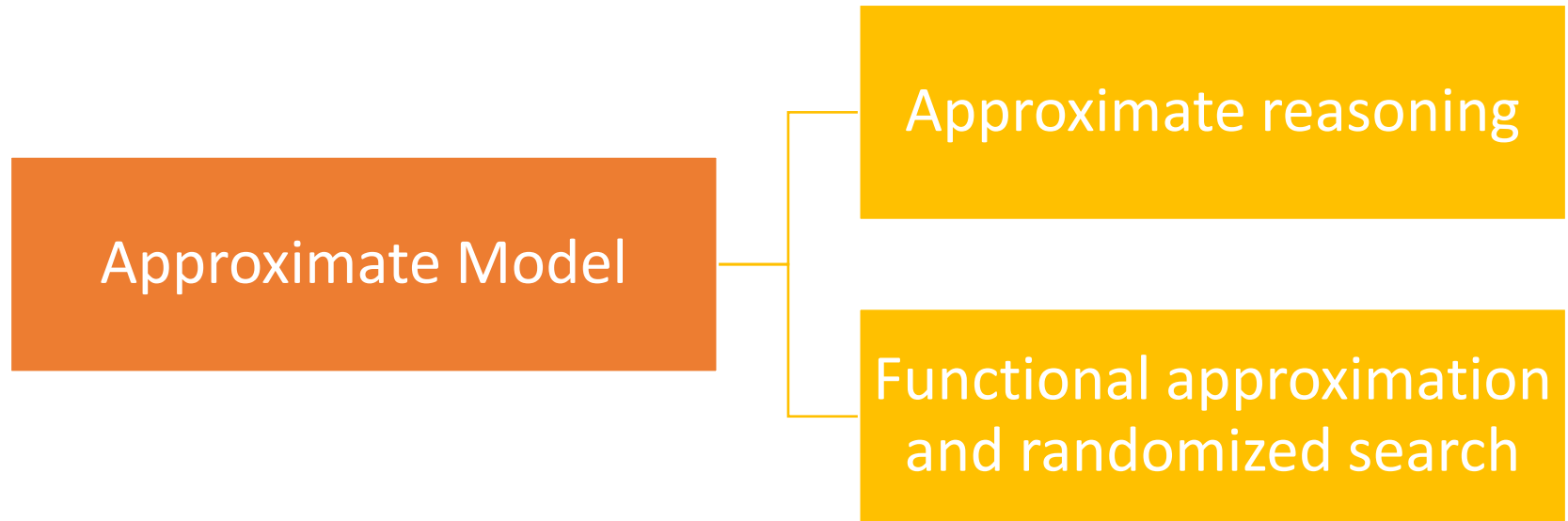
Example: Money allocation problem
(Evolutionary computing)

Examples of soft computing



Example: Robot
(Fuzzy logic)

Soft Computing



How soft computing?

- How a **student** learns from a **teacher**?
 - Teacher asks questions and tell answer then
 - Teacher puts question and hint answers and asks whether the answers are correct or not.
 - Student thus learns a topic and store in his memory.
 - Based on these knowledge he solves new problems.
- This is way how the human brain works.
- Based on this concept **Artificial neural network** is used to solve problems

How soft computing?

- How **world** selects the best?
 - It starts with a population (random).
 - Reproduces another population (next generation).
 - Rank the population and selects the superior individuals.
- **Genetic algorithm** is based on this natural phenomena.
 - Populations is synonymous to solutions.
 - Selection of superior solution is synonymous to exploring the optimal solution.

How soft computing?

- How doctor treats his patient?
 - Doctor asks the patient about suffering.
 - Doctor find the symptoms of diseases
 - Doctor prescribed tests and medicines
- This is exactly how **fuzzy logic** works.
 - Symptoms are correlated with diseases with uncertainty.
 - Doctors prescribes tests/medicines **fuzzily**.

Hard computing vs Soft computing

Hard Computing	Soft Computing
• It requires a precisely stated analytical model and often a lot computation time	• It is tolerant of imprecision, uncertainty, partial truth, and approximation.
• It is based on binary logic, crisp systems, numerical analysis and crisp software	• It is based on fuzzy logic, neural nets and probabilistic reasoning
• It has characteristics of precision and categoricity	• It has characteristics of approximation and dispositionality

Hard computing vs Soft computing

Hard Computing	Soft Computing
• It is deterministic	• It incorporates stochasticity
• It requires exact input data	• It can deal with ambiguous and noisy data
• It is strictly sequential	• It allows parallel computations
• It produces precise answers	• It can yield approximate answers

Applications of Soft Computing

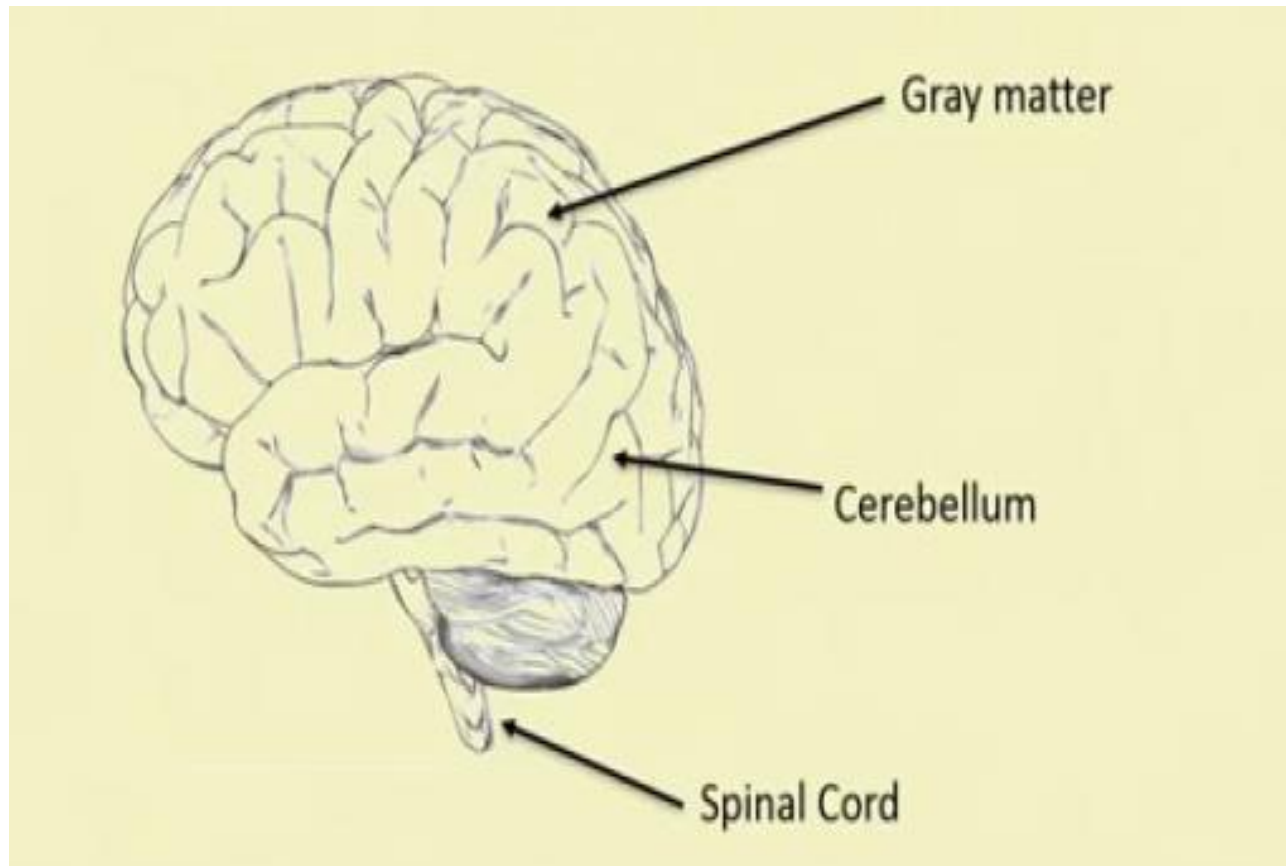


Application Scope of Neural Networks

- Air traffic control
- Animal behavior
- Appraisal and valuation of property, etc.,
- Betting on horse races, stock markets
- Criminal sentencing
- Complex physical and chemical process
- Data mining, cleaning and validation
- Direct mail advertisers
- Echo patterns
- Economic modeling
- Employee hiring
- Expert consultants
- Fraud detection
- Hand writing and typewriting
- Lake water levels
- Machinery controls
- Medical diagnosis
- Music composition

Biological Neurons and its Structure

Brain: Centre of the Nervous system

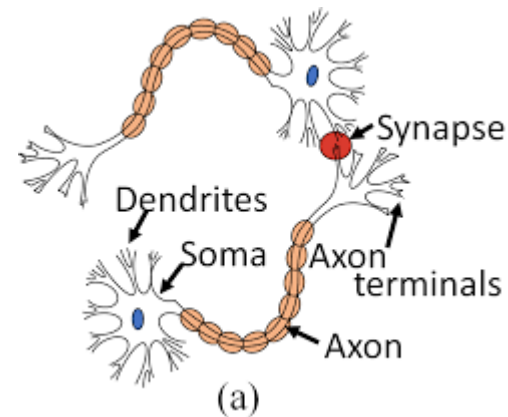


Biological nervous system

- Biological nervous system is most important part of many living things in particular, human beings.
- There is a part called **brain** at the center of human nervous system
- In fact, any biological nervous system consists of a large number of interconnected processing units called **neurons** .
- Each neuron is approximately 10μ long and they can operate in **parallel**.
- Typically the human brain consists of 10^{11} neuron communicating with other with help of **electrical impulses**.

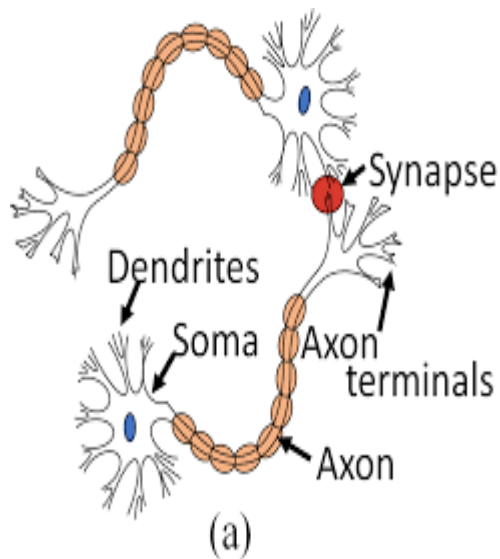
Neuron: Basic unit of nervous system

- **Dendrite** : A bush of very thin fibre.
- **Axon** : A long cylindrical fibre.
- **Soma** : It is also called a cell body, and just like as a nucleus of cell.
- **Synapse** : It is a junction where axon makes contact with the dendrites of neighbouring dendrites.



**Schematic of a biological neuron.
Different parts of it**

Neuron and its working



Dendrite: receives signals from other neurons

Synapse: point of connection with other neurons

Soma: process the information

Axon: Transmit the output of this neuron

Neuron and its working

- There is a chemical in each neuron called neurotransmitter.
- A signal (also called sense) is transmitted across by this chemical.
- That is, all inputs from other neuron arrive to neuron through **dendrites**.
- These signals are accumulated at the synapse of the neuron and then serve as output to be transmitted through the neuron.
- An action may produce an electrical impulse which is usually last for about a **second**
- Note that pulse generated due to an incoming signal and all signal may not produce pulses in axon unless it crosses a **threshold value**
- Also note that action signal in axon of an neuron is **commutative** signal arrive at dendrites which summed up at soma.

Biological Neuron – technical terms(1)

- **Dendrite** (Input): receives signals from other neurons
- **Soma** (cell body – processing unit): sums up all signals. It also consists of threshold unit.
- **Synapse** (weighted connections): The point of interconnection of one neuron with other neurons. The amount of signal transmitted depends upon the strength (synaptic weight) of the connection.

Connection can be **Inhibitory** (decreasing strength) or **Excitatory** (increasing strength) in nature.

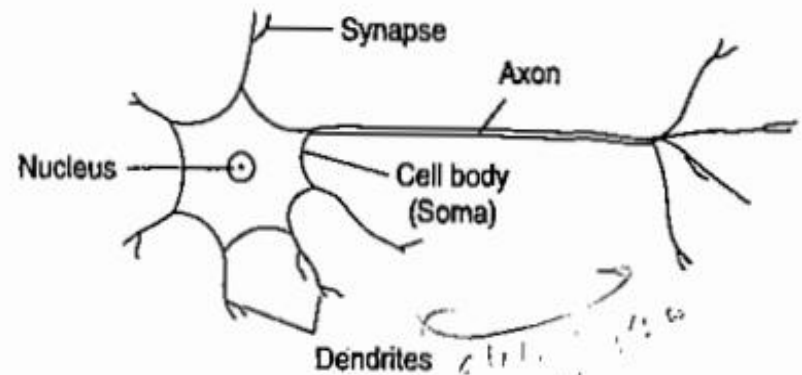


Figure 2-4 Schematic diagram of a biological neuron.

Biological Neuron – technical terms(2)

- **Axon** (output): when the sum reaches to the threshold value, neuron fires and generates an output signal.

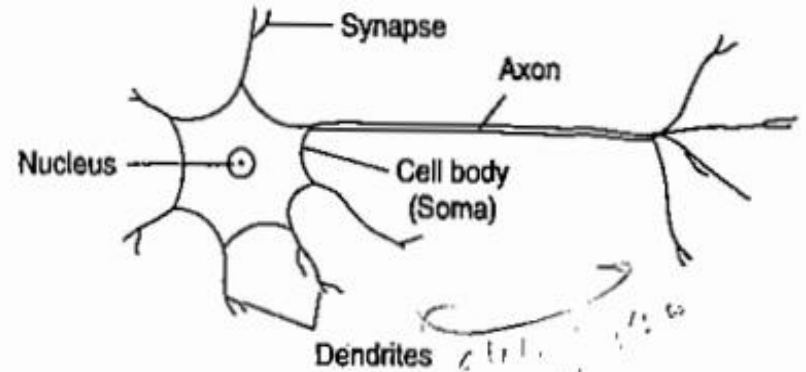


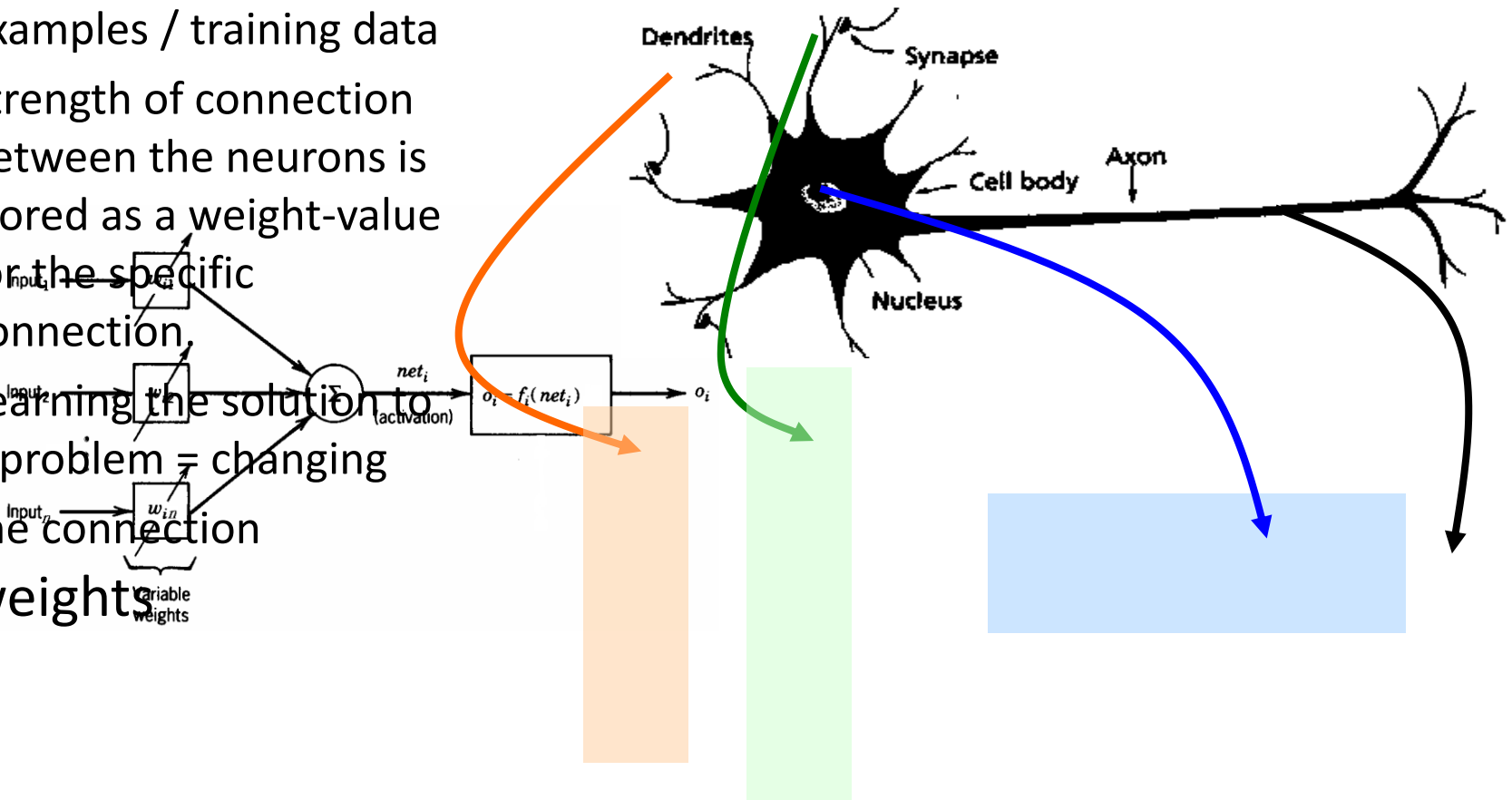
Figure 2-4 Schematic diagram of a biological neuron.

Artificial Neurons

- ANN is an information processing system that has certain performance characteristics in common with biological nets.
- Several key features of the processing elements of ANN are suggested by the properties of biological neurons:
 1. *The processing element receives many signals.*
 2. *Signals may be modified by a weight at the receiving synapse.*
 3. *The processing element sums the weighted inputs.*
 4. *Under appropriate circumstances (sufficient input), the neuron transmits a single output.*
 5. *The output from a particular neuron may go to many other neurons.*

Artificial Neurons

- From experience: examples / training data
- Strength of connection between the neurons is stored as a weight-value for the specific connection
- Learning the solution to a problem = changing the connection weights

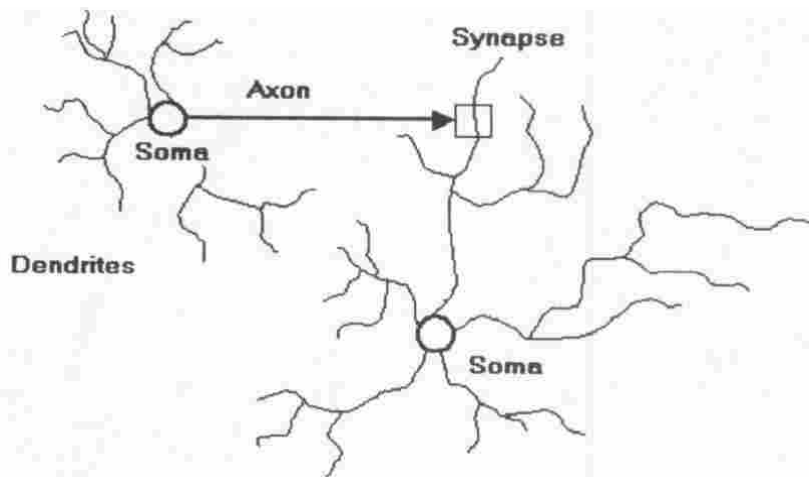


An artificial neuron

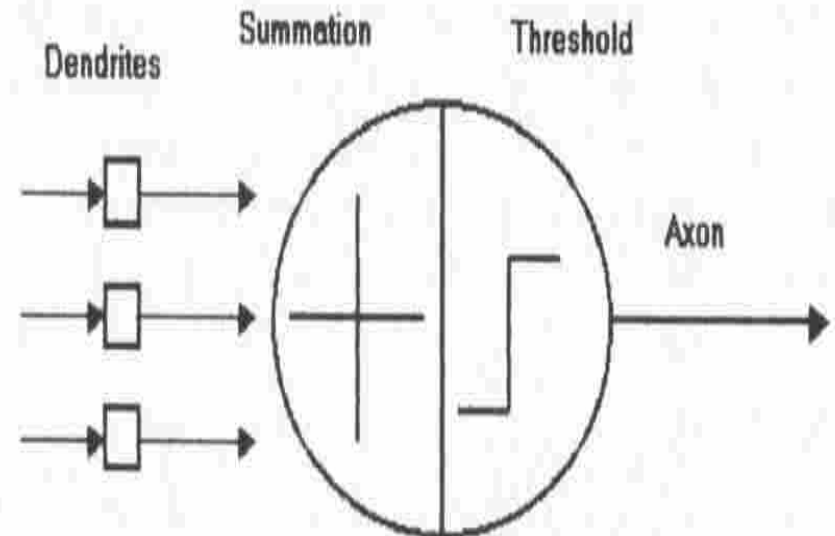
Artificial Neurons

- ANNs have been developed as generalizations of mathematical models of neural biology, based on the assumptions that:
 1. Information processing occurs at many simple elements called neurons.
 2. Signals are passed between neurons over connection links.
 3. Each connection link has an associated weight, which, in typical neural net, multiplies the signal transmitted.
 4. Each neuron applies an activation function to its net input to determine its output signal.

Artificial Neuron



Four basic components of a human biological neuron

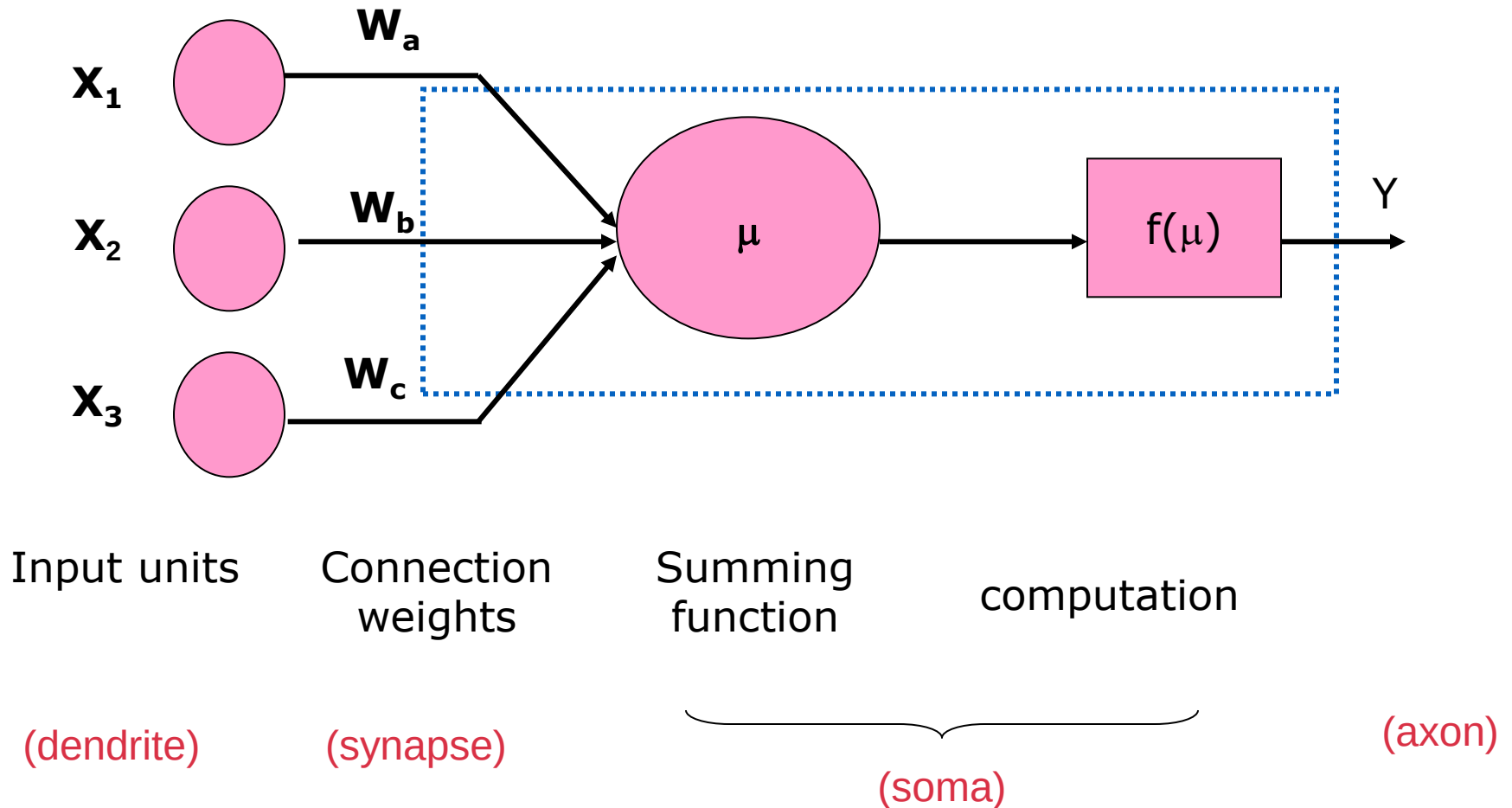


The components of a basic artificial neuron

Terminology Relation Between Biological And Artificial Neuron

Biological Neuron	Artificial Neuron
Cell	Neuron
Dendrites	Weights or interconnections
Soma	Net input
Axon	Output

Model Of A Neuron



- A neural net consists of a large number of simple processing elements called neurons, units, cells or nodes.
- Each neuron is connected to other neurons by means of directed communication links, each with associated weight.
- The weight represent information being used by the net to solve a problem.

- Each neuron has an internal state, called its activation or activity level, which is a function of the inputs it has received. Typically, a neuron sends its activation as a signal to several other neurons.
- It is important to note that a neuron can send only one signal at a time, although that signal is broadcast to several other neurons.

Biological vs Artificial Neuron

Criteria	BN	AN
Speed of operation	Execution time is the order of milliseconds	Execution time is the order of nanoseconds (faster)
Data Processing	Parallel processing	Large no of instructions processed in a sequential mode
Data Storage	Synaptic interconnections – information is stored in the strength of interconnection. Not replaceable	Specifically allotted memory allocations. Replaceable
Storage capacity	Unlimited	Limited
Data retrieval	Data loss may occur due to memory failure	Easy at any time

Biological vs Artificial Neuron

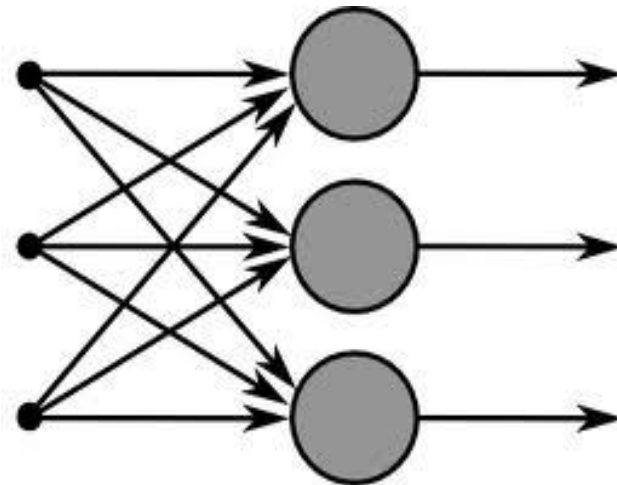
Criteria	BNN	ANN
Size - No. of neurons	10^{11}	Comparatively less
Complexity	More complex than ANN	Do not involve much computational neurons so, difficult to perform complex computation
Tolerance	Damage to one of neuron cell doesn't cause much damage to nervous system	One of interconnection link fails lead to network failure
Control System	No central control. The neuron acts based on the information locally available and transmits its output to other neurons.	Continuously monitored by control unit.

Basic models of ANN

- Models are based on three entities
 - *The model's synaptic interconnections.*
 - *The training or learning rules adopted for updating and adjusting the connection weights.*
 - *Their activation functions*
- The *arrangement of neurons to form layers* and the *connection pattern formed within and between layers* is called the ***network architecture.***
- Five types:
 - *Single layer feed forward network*
 - *Multilayer feed-forward network*
 - *Single node with its own feedback*
 - *Single-layer recurrent network*
 - *Multilayer recurrent network*

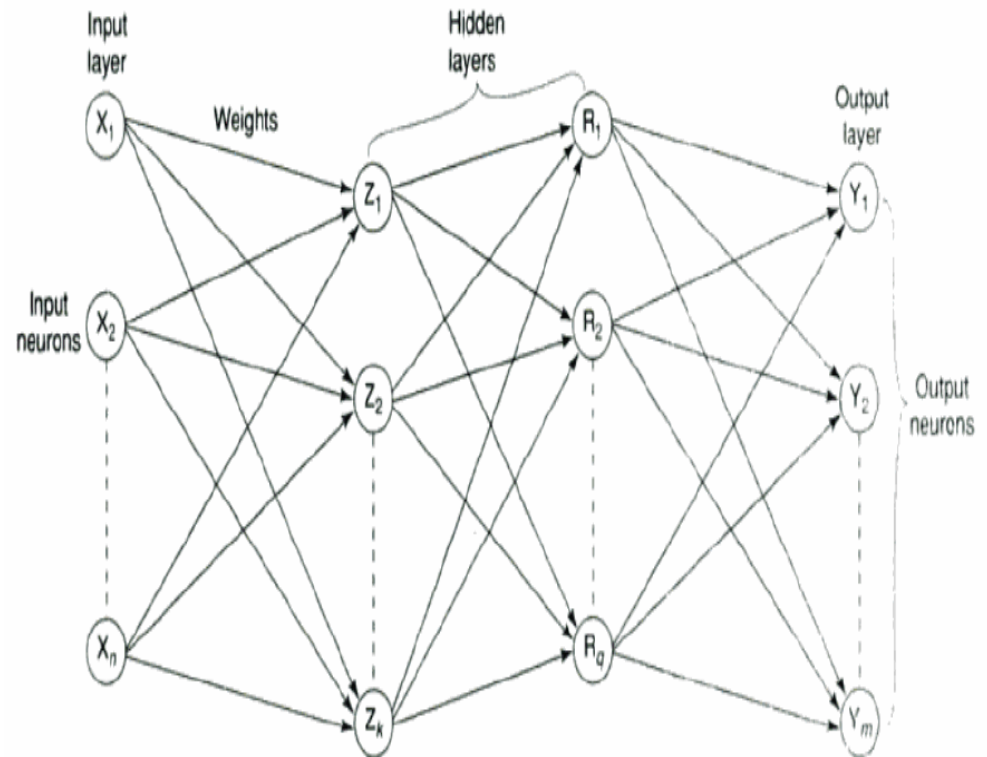
Single layer Feed- Forward Network

- Layer is formed by taking processing elements and combining it with other processing elements.
- Input and output are linked with each other
- Inputs are connected to the processing nodes with various weights, resulting in series of outputs one per node.



Multilayer feed-forward network

- Formed by the interconnection of several layers.
- Input layer receives input and buffers input signal.
- Output layer generated output.
- Layer between input and output is called *hidden layer*.
- Hidden layer is internal to the network.
- Zero to several hidden layers in a network.
- More the hidden layer, more is the complexity of network, but efficient output is produced.



Feed back network

- If no neuron in the output layer is an input to a node in the same layer / proceeding layer – **feed forward network**.
- If outputs are directed back as input to the processing elements in the same layer/proceeding layer – **feedback network**.
- If the output are directed back to the input of the same layer then it is **lateral feedback**.
- **Recurrent networks** are networks with feedback networks with closed loop.
- Fig 2.8 (A) –simple recurrent neural network having a single neuron with feedback to itself.
- Fig 2.9 – single layer network with feedback from output can be directed to processing element itself or to other processing element/both.

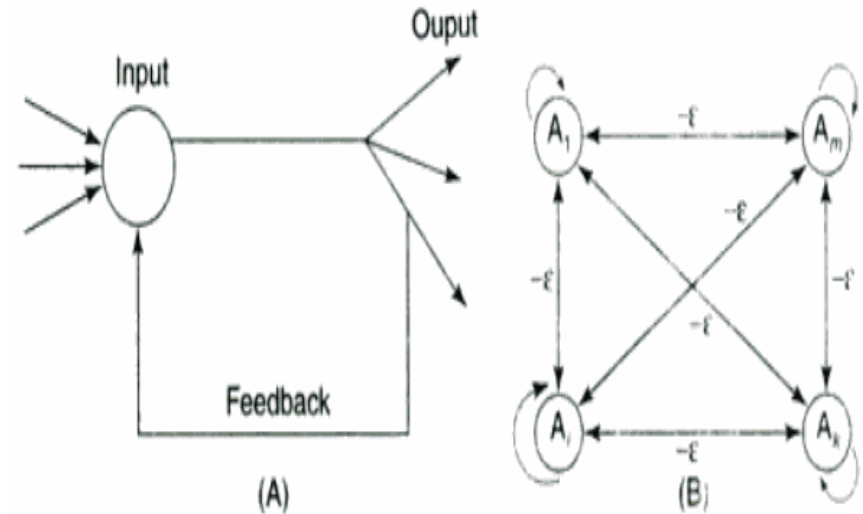


Figure 2-8 (A) Single node with own feedback. (B) Competitive nets.

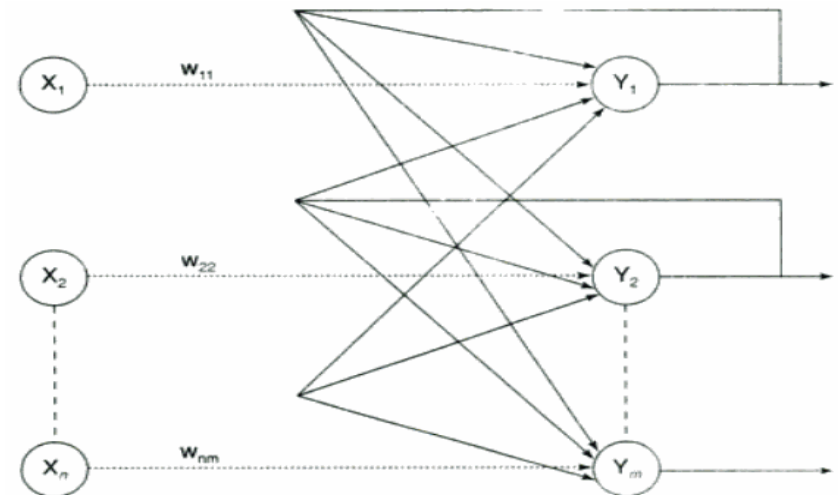
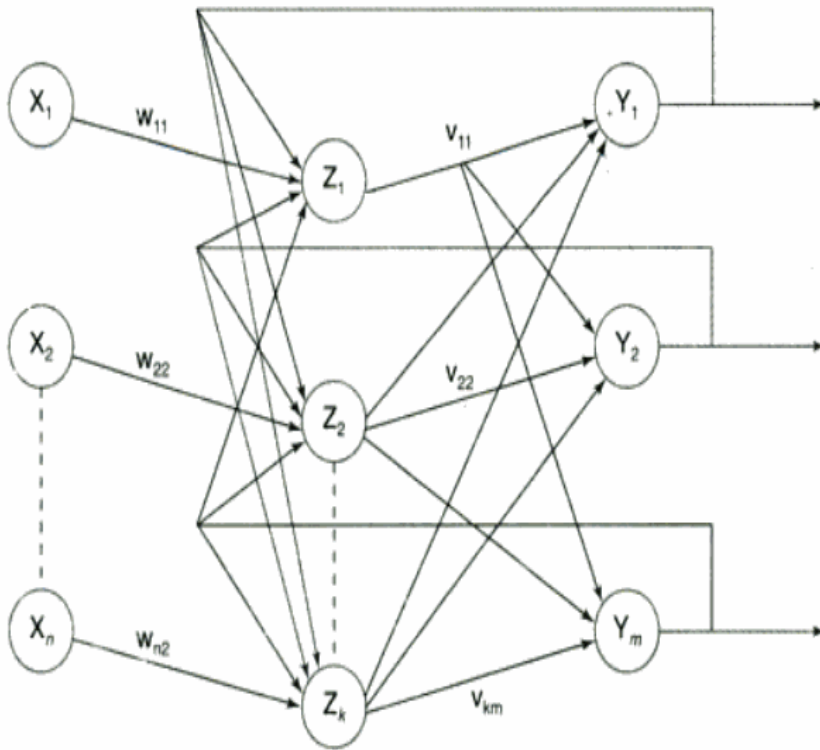


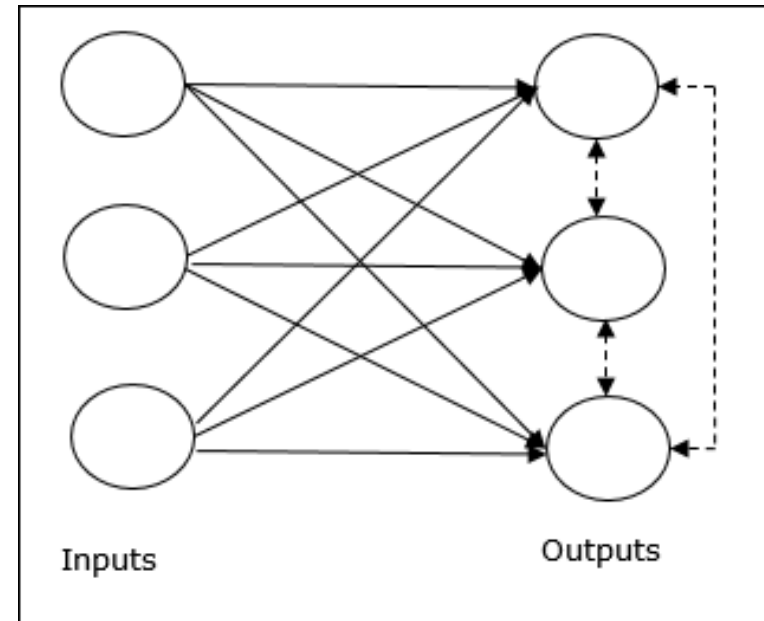
Figure 2-9 Single-layer recurrent network.



- Processing element output can be directed back to the nodes in the preceding layer, forming a ***multilayer recurrent network***.
- Processing element output can be directed to processing element itself or to other processing element in the same layer.

Competitive network

- This network is just like a single layer feed-forward network having feedback connection between the outputs. The connections between the outputs are inhibitory type, which is shown by dotted lines, which means the competitors never support themselves..



Artificial Neural Network Terminologies

Weights

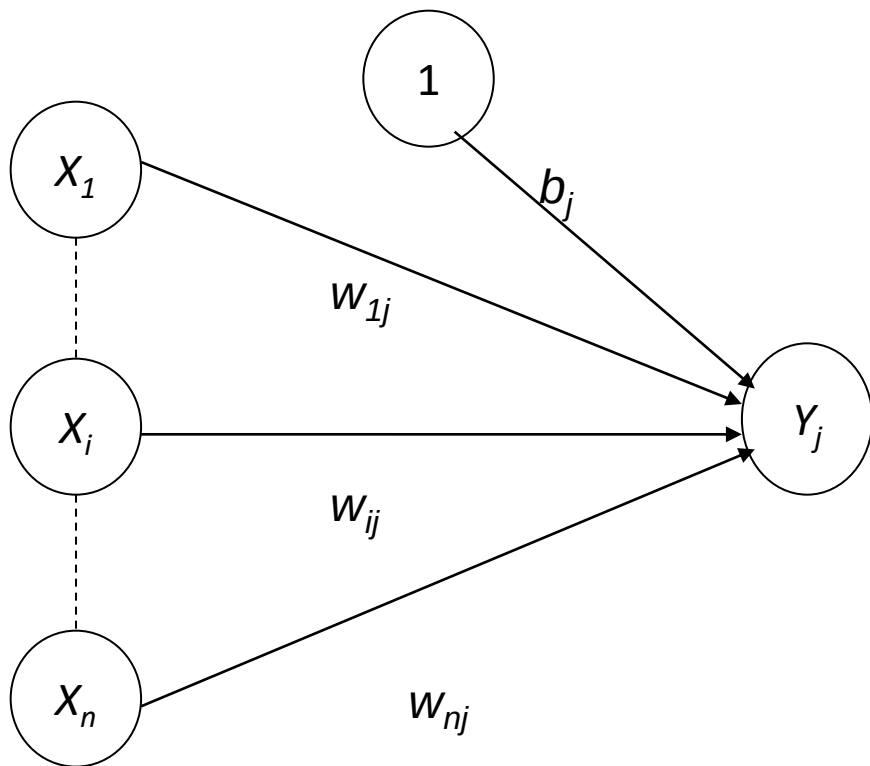
- Each neuron is connected to every other neuron by means of directed links
- Links are associated with weights
- Weights contain information about the input signal and is represented as a matrix
- Weight matrix also called connection matrix

Weight matrix

$$W = \begin{bmatrix} w_1^T \\ w_2^T \\ w_3^T \\ \vdots \\ \vdots \\ \vdots \\ w_n^T \end{bmatrix} = \begin{bmatrix} w_{11} w_{12} w_{13} \cdots w_{1m} \\ w_{21} w_{22} w_{23} \cdots w_{2m} \\ \dots\dots\dots \\ \dots\dots\dots \\ w_{n1} w_{n2} w_{n3} \cdots w_{nm} \end{bmatrix}$$

Weights contd...

- w_{ij} is the weight from processing element "i" (source node) to processing element "j" (destination node)



$$y_{inj} = \sum_{i=0}^n x_i w_{ij}$$
$$= x_0 w_{0j} + x_1 w_{1j} + x_2 w_{2j} + \dots + x_n w_{nj}$$

$$= w_{0j} + \sum_{i=1}^n x_i w_{ij}$$

$$y_{inj} = b_j + \sum_{i=1}^n x_i w_{ij}$$

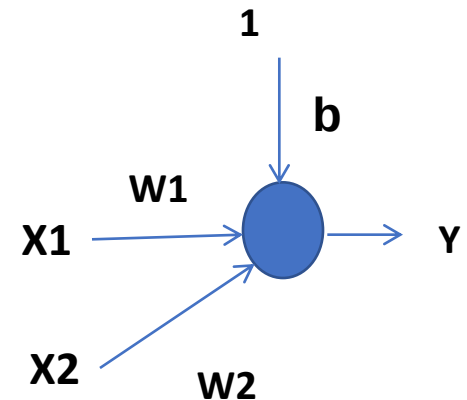
Bias

- Bias has an impact in calculating net input.
- Bias is included by adding x_0 to the input vector x .
- The net output is calculated by

$$y_{in_j} = \sum_{i=0}^n x_i w_{ij}$$

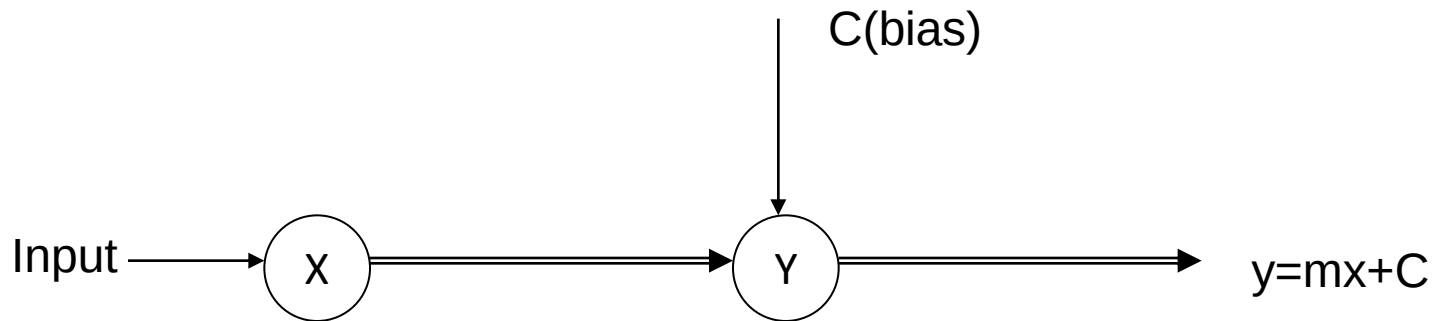
$$y_{in_j} = b_j + \sum_{i=0}^n x_i w_{ij}$$

- The bias is of two types
 - Positive bias
 - Increase the net input
 - Negative bias
 - Decrease the net input



Why Bias is required?

- The relationship between input and output given by the equation of straight line $y=mx+c$



Threshold

- It is a set value based upon which the final output is calculated.
- Calculated net input and threshold is compared to get the network output.
- The activation function of threshold is defined as

$$f(net) = \begin{cases} 1 & \text{if } net \geq \theta \\ -1 & \text{if } net < \theta \end{cases}$$

- where θ is the fixed threshold value

Learning rate

- Denoted by α .
- Used to control the amount of weight adjustment at each step of training
- Learning rate ranging from 0 to 1 determines the rate of learning in each time step i

Other terminologies

- Momentum factor:
 - used for convergence when momentum factor is added to weight updation process.
- Vigilance parameter:
 - Denoted by ρ
 - Used to control the degree of similarity required for patterns to be assigned to the same cluster

Activation functions

- To make work more efficient and for exact output, some force or activation is given. Like that, activation function is applied over the net input to calculate the output of an ANN.
- Information processing of processing element has two major parts: input and output. An integration function (f) is associated with input of processing element.
- Several activation functions are there.

1. Identity function:

- it is a linear function which is defined as

$$f(x) = x \text{ for all } x$$

- The output is same as the input.

2. Binary step function

- it is defined as

$$f(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ 0 & \text{if } x < \theta \end{cases}$$

where θ represents threshold value.

It is used in single layer nets to convert the net input to an output that is binary. (0 or 1)

3. Bipolar step function:

It is defined as

$$f(x) = \begin{cases} 1 & \text{if } x \geq \theta \\ -1 & \text{if } x < \theta \end{cases}$$

where θ represents threshold value. Used in single layer nets to convert the net input to an output that is bipolar (+1 or -1).

4. Sigmoid function

used in Back propagation nets.

Two types:

a) binary sigmoid function

-logistic sigmoid function or unipolar sigmoid function.

-it is defined as

$$f(x) = \frac{1}{1 + e^{-\lambda x}}$$

where λ – steepness parameter.

-The derivative of this function is

$f'(x) = \lambda f(x)[1-f(x)]$. The range of sigmoid function is 0 to 1.

b) Bipolar sigmoid function

$$f(x) = \frac{2}{1 + e^{-\lambda x}} - 1 = \frac{1 - e^{-\lambda x}}{1 + e^{-\lambda x}}$$

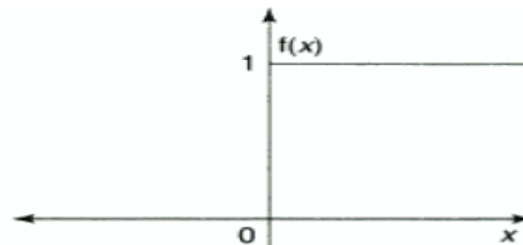
where λ - steepness parameter and the sigmoid range is between -1 and +1.

5. Ramp function

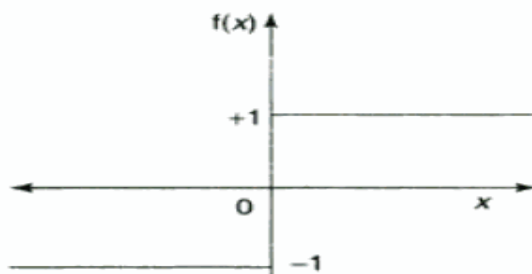
$$f(x) = \begin{cases} 1 & \text{if } x > 1 \\ x & \text{if } 0 \leq x \leq 1 \\ 0 & \text{if } x < 0 \end{cases}$$



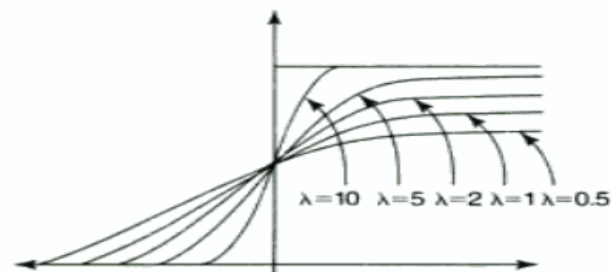
(A)



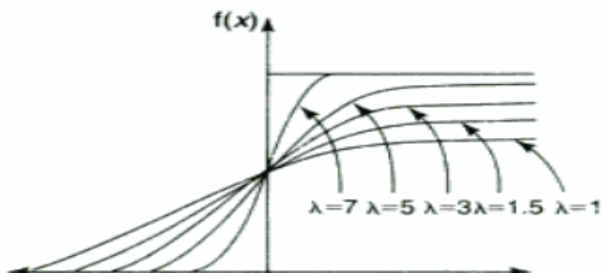
(B)



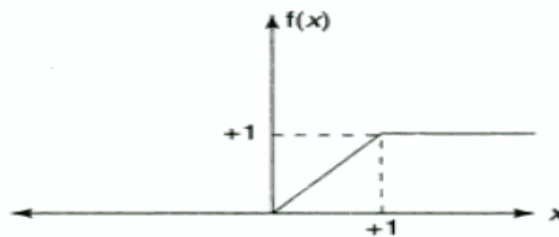
(C)



(D)



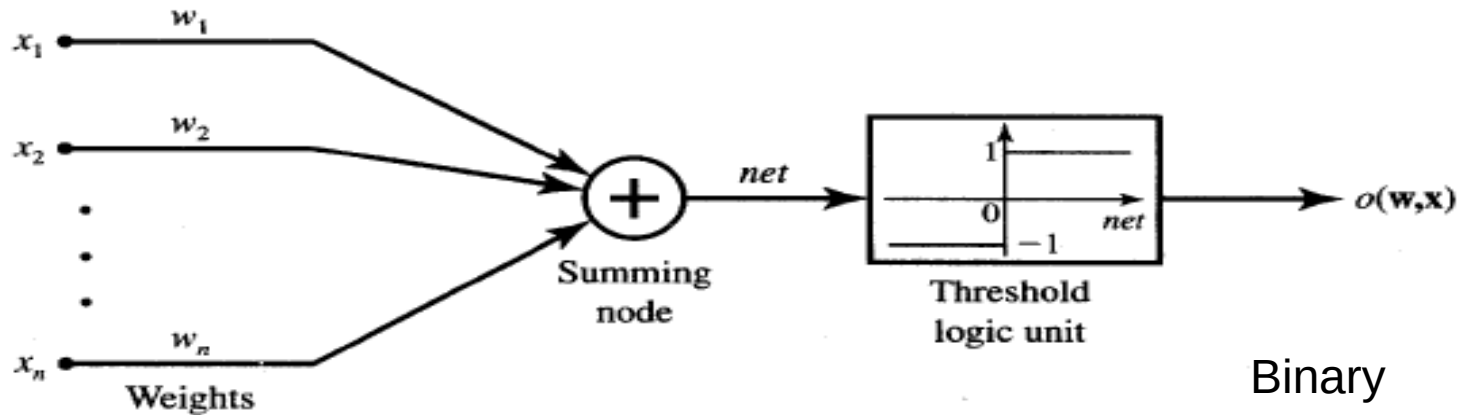
(E)



(F)

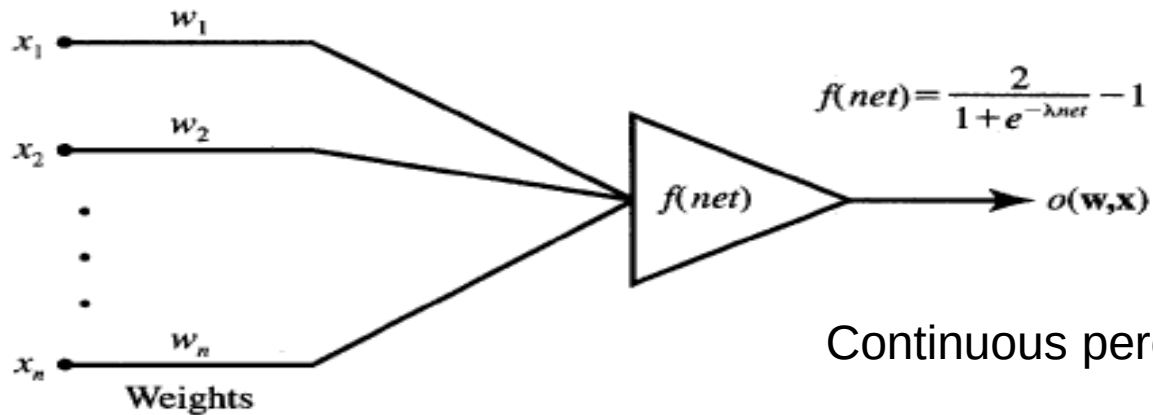
2-15 Depiction of activation functions: (A) identity function; (B) binary step function; (C) bipolar step function; (D) binary sigmoidal function; (E) bipolar sigmoidal function; (F) ramp function.

Common models of neurons



Binary
perceptrons

(a)



Continuous perceptrons

(b)

Learning

Two broad kinds of learning in ANNs is :

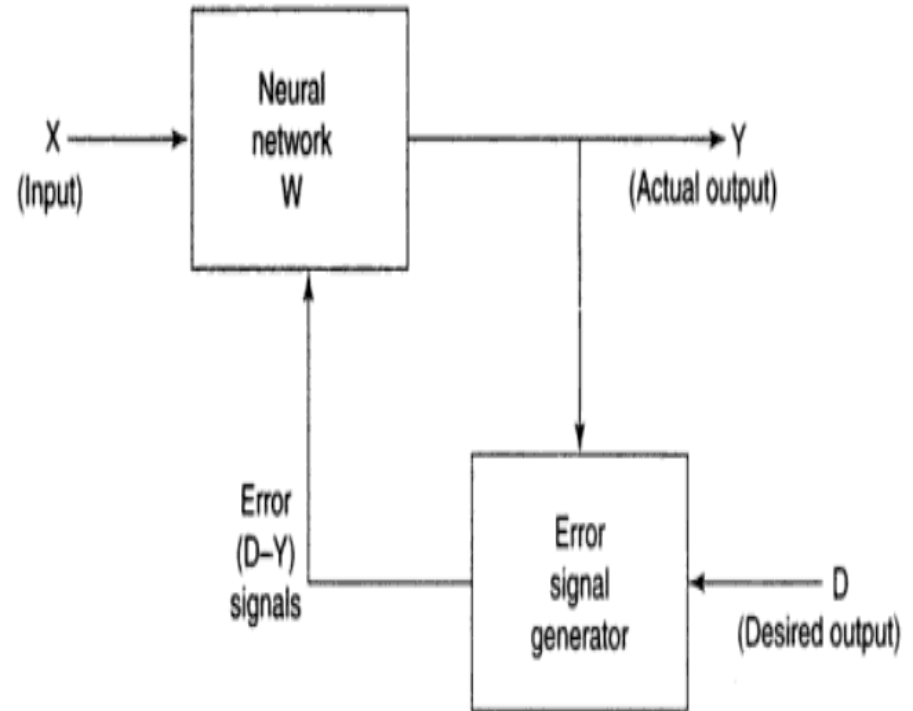
- i) parameter learning – updates connecting weights in a neural net.
- ii) Structure learning – focus on change in the network.

Apart from these, learning in ANN is classified into three categories as

- i) supervised learning
- ii) unsupervised learning
- Iii) reinforcement learning

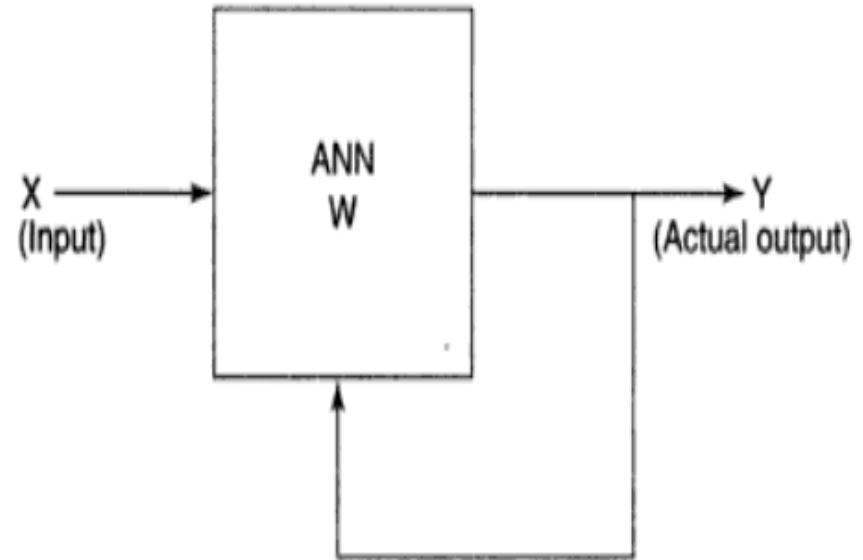
Supervised learning

- Learning with the help of a teacher.
- Example : learning process of a small child.
 - Child doesn't know read/write.
 - Their each & every action is supervised by a teacher
- In ANN, each input vector requires a corresponding target vector, which represents the desired output.
- The input vector along with target vector is called **training pair**.
- The input vector results in output vector.
- The actual output vector is compared with desired output vector.
- If there is a difference means an error signal is generated by the network.
- It is used for adjustment of weights until actual output matches desired output.

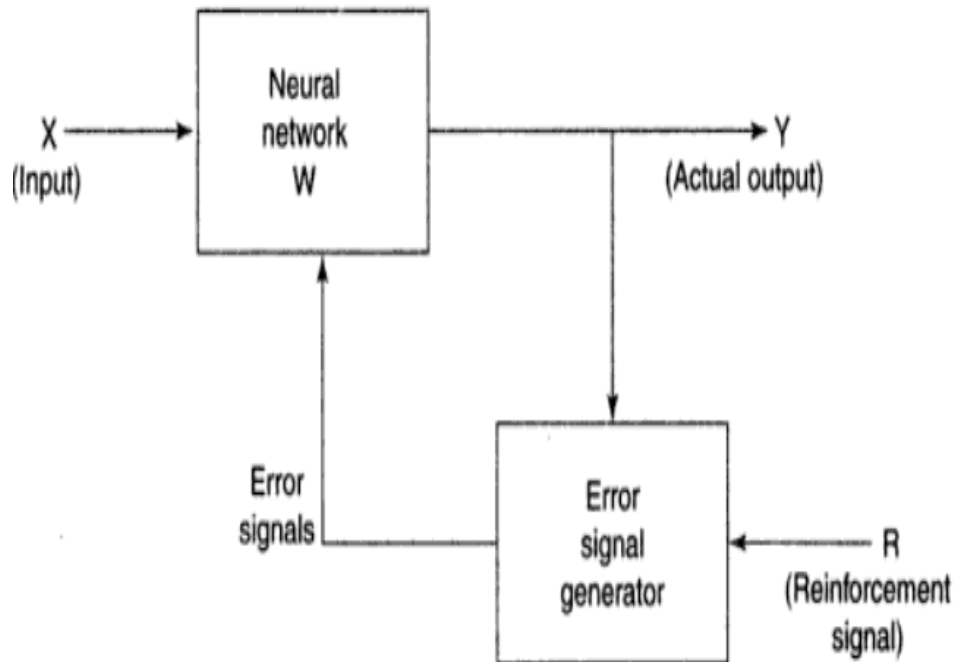


Unsupervised learning

- Learning is performed without the help of a teacher.
- Example: tadpole – learn to swim by itself.
- In ANN, during training process, network receives input patterns and organize it to form clusters.
- From the Fig. it is observed that no feedback is applied from environment to inform what output should be or whether they are correct.
- The network itself discover patterns, regularities, features/ categories from the input data and relations for the input data over the output.
- Exact clusters are formed by discovering similarities & dissimilarities so called as *self – organizing*.

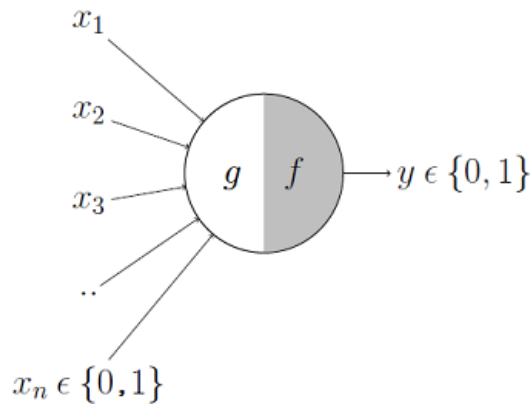


Reinforcement learning



- Similar to supervised learning.
- Learning based on *critic information* is called *reinforcement learning* & the feedback sent is called *reinforcement signal*.
- The network receives some feedback from the environment.
- Feedback is only evaluative.
- The external reinforcement signals are processed in the critic signal generator, and the obtained critic signals are sent to the ANN for adjustment of weights properly to get critic feedback in future.

McCulloch Pitts Neuron



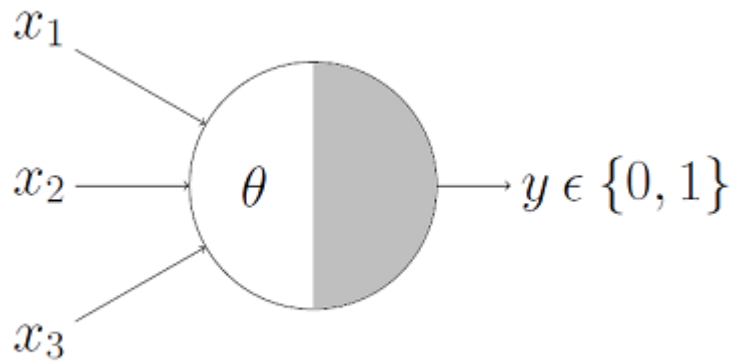
- McCulloch (neuroscientist) and Pitts (logician) proposed a highly simplified computational model of the neuron (1943)
- g aggregates the inputs and the function f takes a decision based on this aggregation
- The inputs can be excitatory or inhibitory
- $y = 0$ if any x_i is inhibitory

$$g(x_1, x_2, x_3, \dots, x_n) = g(\mathbf{x}) = \sum_{i=1}^n x_i$$

$$y = f(g(\mathbf{x})) = \begin{cases} 1 & \text{if } g(\mathbf{x}) \geq \theta \\ 0 & \text{if } g(\mathbf{x}) < \theta \end{cases}$$

- θ is called the thresholding parameter
- This is called Thresholding Logic

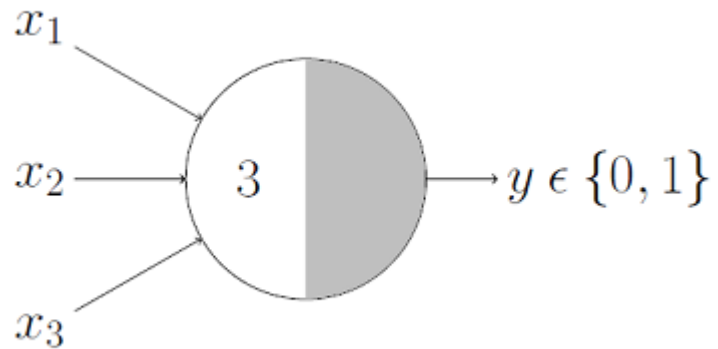
Boolean Functions Using M-P Neuron



This representation just denotes that, for the boolean inputs x_1 , x_2 and x_3 if the $g(x)$ i.e., $\text{sum} \geq \theta$, the neuron will fire otherwise, it won't.

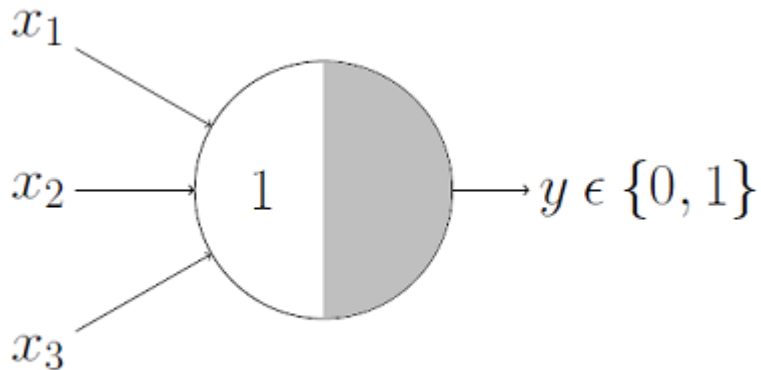
Concise representation of M-P neuron

AND Function



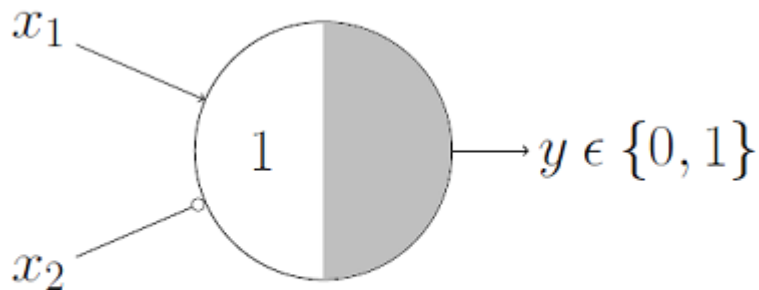
AND function neuron would only fire when ALL the inputs are ON i.e., $g(\mathbf{x}) \geq 3$.

OR Function



- OR function neuron would fire if ANY of the inputs is ON i.e., $g(x) \geq 1$

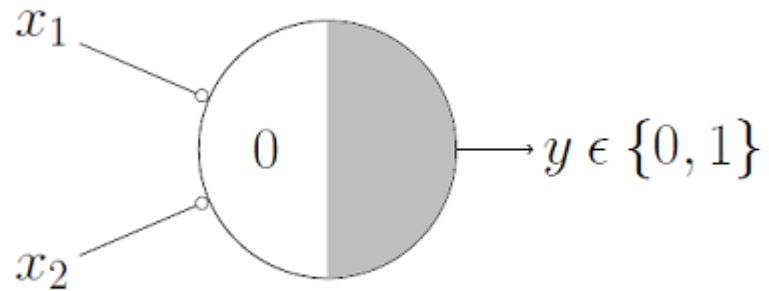
A Function With An Inhibitory Input



$$x_1 \text{ AND } !x_2^*$$

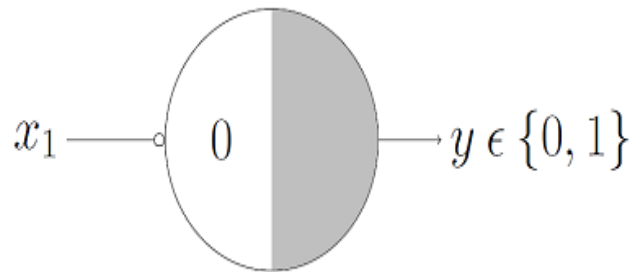
- we have an inhibitory input i.e., x_2 so whenever x_2 is 1, the output will be 0.
- $x_1 \text{ AND } !x_2$ would output 1 only when x_1 is 1 and x_2 is 0 so the threshold parameter should be 1.

NOR Function



For a NOR neuron to fire, we want ALL the inputs to be 0 so the thresholding parameter should also be 0 and we take them all as inhibitory input

NOT Function



For a NOT neuron, 1 outputs 0 and 0 outputs 1. So we take the input as an inhibitory input and set the thresholding parameter to 0

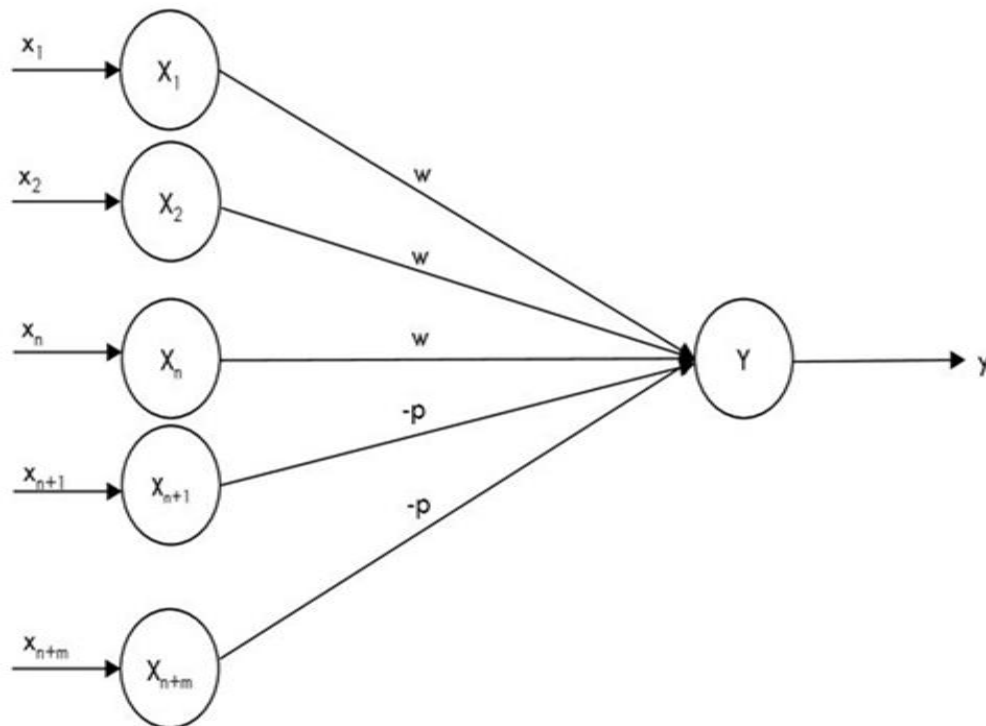
McCulloch Pitts Neuron model (1)

- Also called as M-P Neuron model.
- First computational model proposed by Warren McCulloch and Walter Pitts in 1943.
- M-P neurons are connected by directed weighted path
- Activation function of M-P neuron is binary i.e. at any time step the neuron may fire or not
- Weights associated with communication link are excitatory (weight positive) or inhibitory (weight is negative)

McCulloch Pitts Neuron model (2)

- All excitatory weights(+ve) will have the same value.
- There is a fixed threshold for each neuron and if the net input to the neurons are greater than the threshold than the neuron fires.
- Any non-zero inhibitory weight (-ve) will prevent the neuron from fired.
- No particular training algorithm
- M-P neurons are widely used in logic functions (AND, OR etc.)

MCCULLOCH-PITTS NEURON: ARCHITECTURE (MP NEURON MODEL)



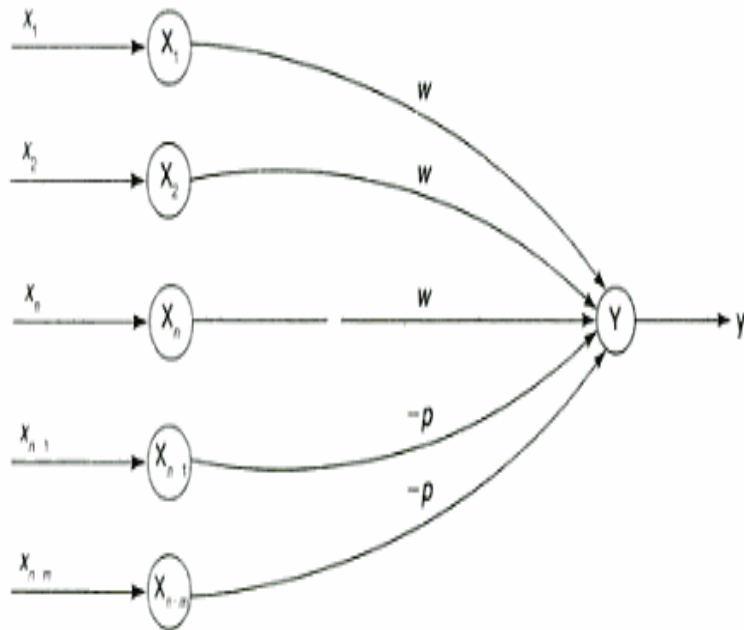
McCULLOCH-PITTS NEURON

Activation function for MP neuron is

$$f(y) = \begin{cases} 1 & \text{if } y \geq \theta \\ 0 & \text{if } y < \theta \end{cases}$$

And

$$\theta > nw - p$$



- A simple M-P neuron is shown in the figure.
- It is excitatory with weight ($w > 0$) / inhibitory with weight $-p$ ($p < 0$).
- In the Fig., inputs from x_1 to x_n possess excitatory weighted connection and X_{n+1} to x_{n+m} has inhibitory weighted interconnections.
- Since the firing of neuron is based on threshold, activation function is defined as

$$f(y_{in}) = \begin{cases} 1 & \text{if } y_{in} \geq \theta \\ 0 & \text{if } y_{in} < \theta \end{cases}$$

- For inhibition to be absolute, the threshold with the activation function should satisfy the following condition:

$$\theta > \sum w_i - p$$

- Output will fire if it receives “ k ” or more excitatory inputs but no inhibitory inputs where

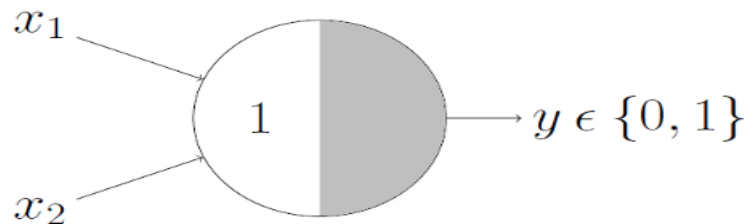
$$\sum_{i=1}^k w_i \geq \theta > \sum_{i=1}^{k-1} w_i$$

- The M-P neuron has no particular training algorithm.

- An analysis is performed to determine the weights and the threshold.

- It is used as a building block where any function or phenomenon is modeled based on a logic function.

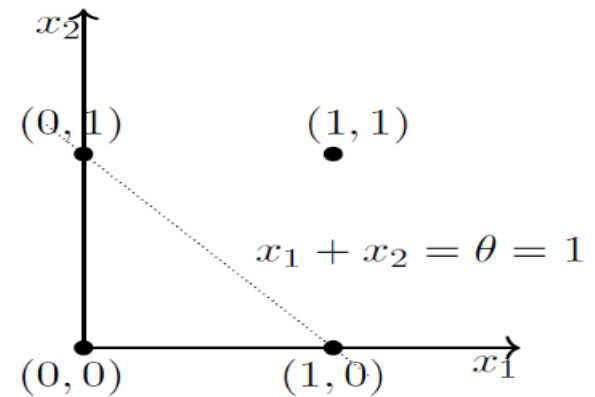
Geometrical interpretation of OR function MP-neuron



OR function

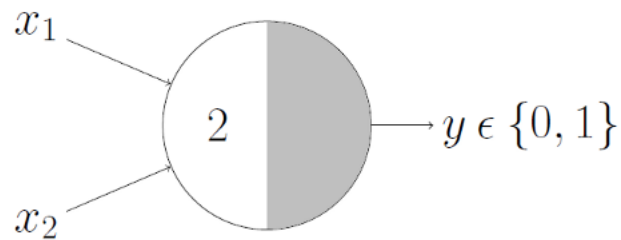
$$x_1 + x_2 = \sum_{i=1}^2 x_i \geq 1$$

- A single MP neuron splits the input points (4 points for 2 binary inputs) into two halves
- Points lying on or above the line $\sum_{i=1}^n x_i - \theta = 0$ and points lying below this line.



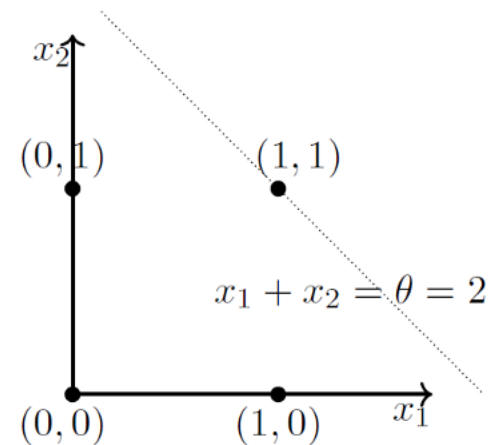
- all inputs which produce an output 0 will be on one side ($\sum_{i=1}^n x_i < \theta$) of the line and all inputs which produce an output 1 will lie on the other side ($\sum_{i=1}^n x_i > \theta$) of this line

Geometrical interpretation of AND function MP-neuron

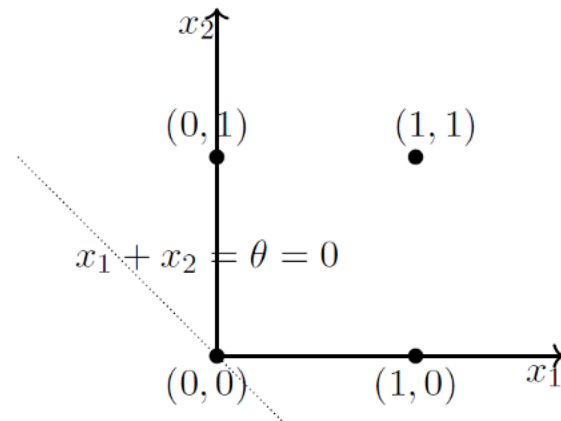
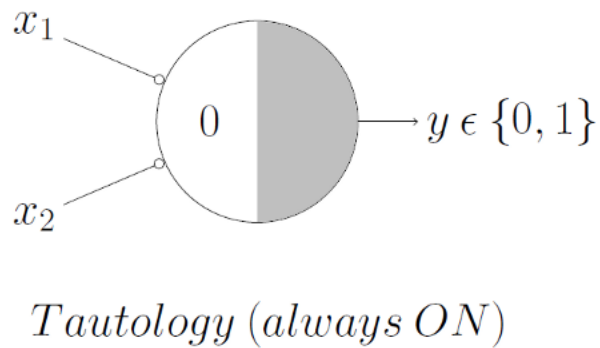


AND function

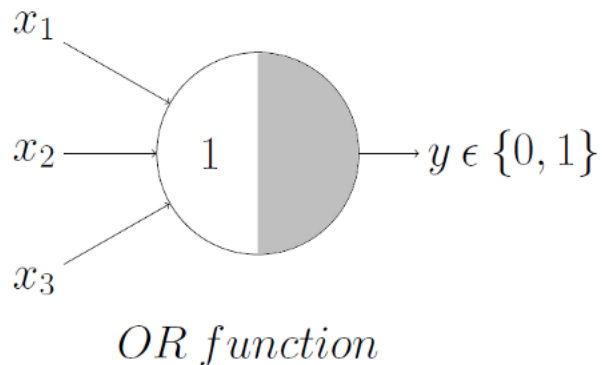
$$x_1 + x_2 = \sum_{i=1}^2 x_i \geq 2$$



Geometrical interpretation of Tautology MP-neuron

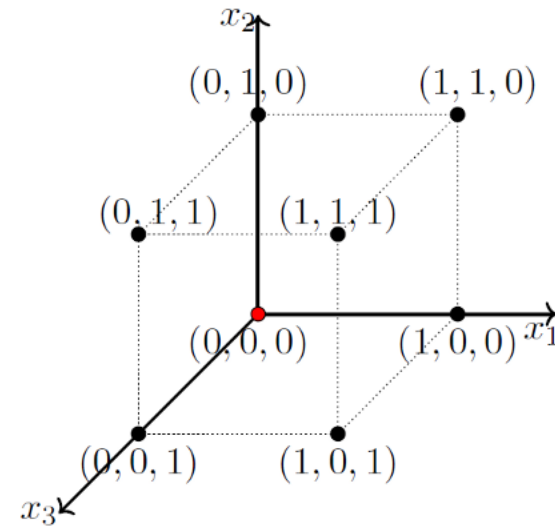


Geometrical interpretation of OR function MP-neuron with 3 inputs

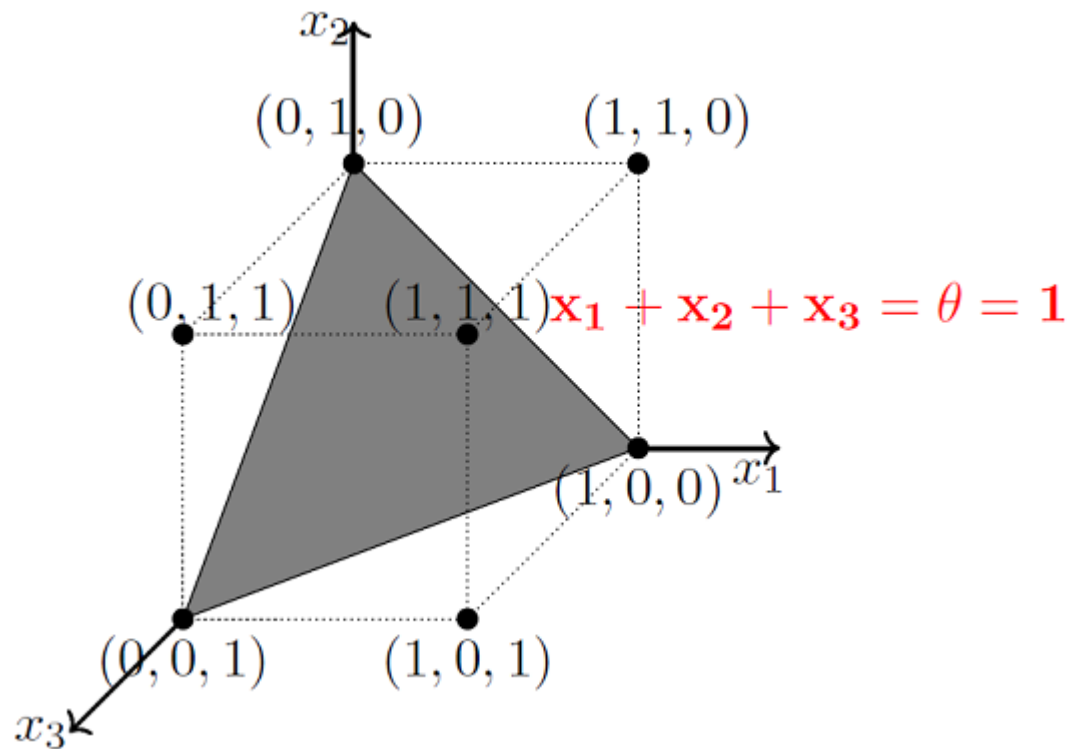


OR function

$$x_1 + x_2 + x_3 = \sum_{i=1}^3 x_i \geq 1$$



Geometrical interpretation of OR function MP-neuron with 3 inputs

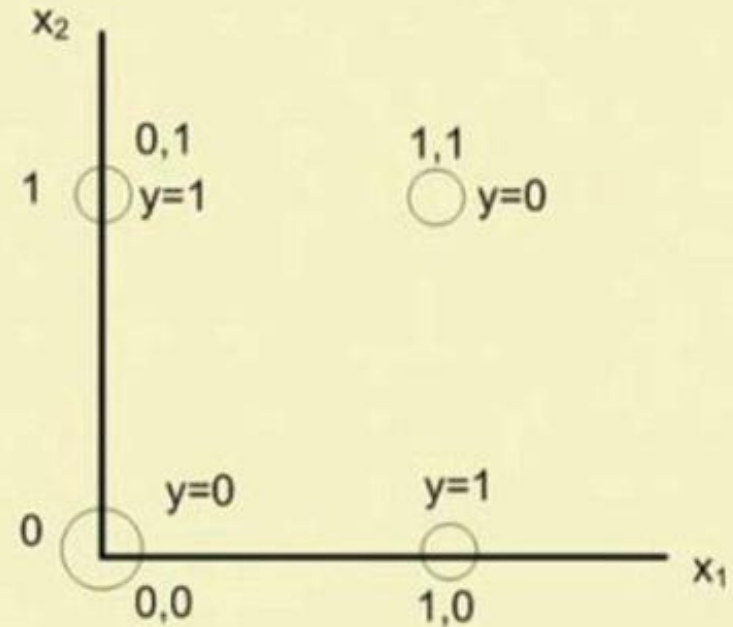


For the OR function, we want a plane such that the point $(0,0,0)$ lies on one side and the remaining 7 points lie on the other side of the plane

XOR PROBLEM IS LINEARLY NON-SEPARABLE

x_1	x_2	Output (y)
0	0	0
0	1	1
1	0	1
1	1	0

XOR Problem

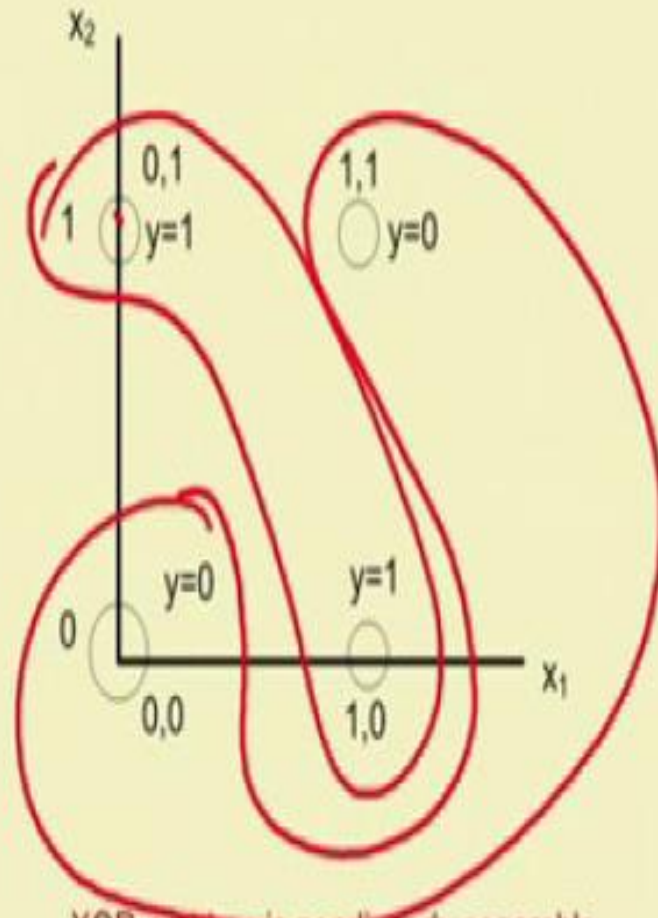


XOR-problem is non-linearly separable

XOR problem is linearly non-separable

x_1	x_2	Output (y)
0	0	0
0	1	1
1	0	1
1	1	0

XOR Problem



XOR-problem is non-linearly separable

Thus..

- A single McCulloch Pitts Neuron can be used to represent boolean functions which are linearly separable
- **Linear separability (for boolean functions)** : There exists a line (plane) such that all inputs which produce a 1 lie on one side of the line (plane) and all inputs which produce a 0 lie on other side of the line (plane)

Limitation of MP-neuron

- What about non-boolean (say, real) inputs?
- Do we always need to hand code the threshold?
- Are all inputs equal? What if we want to assign more importance to some inputs?
- What about functions which are not linearly separable? Say XOR function.

Linear separability

- It is a concept wherein the separation of the input space into regions is based on whether the network response is positive or negative.
- A decision line is drawn to separate positive or negative response.
- The decision line is also called as *decision-making line* or *decision-support line* or *linear-separable line*.
- The net input calculation to the output unit is given as

$$y_{in} = b_j + \sum_{i=1}^n x_i w_i$$

- The region which is called as *decision boundary* is determined by the relation $b + \sum_{i=1}^n x_i w_i = 0$

Contd..

- Linear separability of the network is based on decision-boundary line.
- If there exists weights(bias) for which training input vectors have positive response (+1 ve) input lie on one side of line and other having negative response(-ve) lie on other side of line, we conclude the problem is linearly separable

Contd..

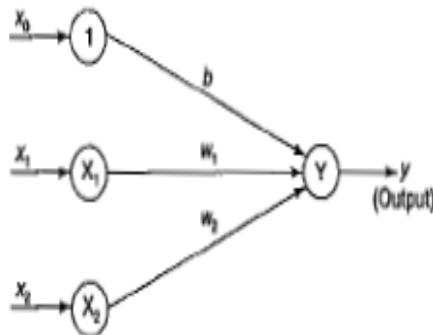


Figure 2-19 A single-layer neural net.

Net input of the network

$$Y_{in} = b + x_1 w_1 + x_2 w_2$$

Separating line for which the boundary lies between the values of x_1 and x_2

$$b + x_1 w_1 + x_2 w_2 = 0$$

If weight of w_2 is not equal to 0 then we get

$$x_2 = -w_1/w_2 - b/w_2$$

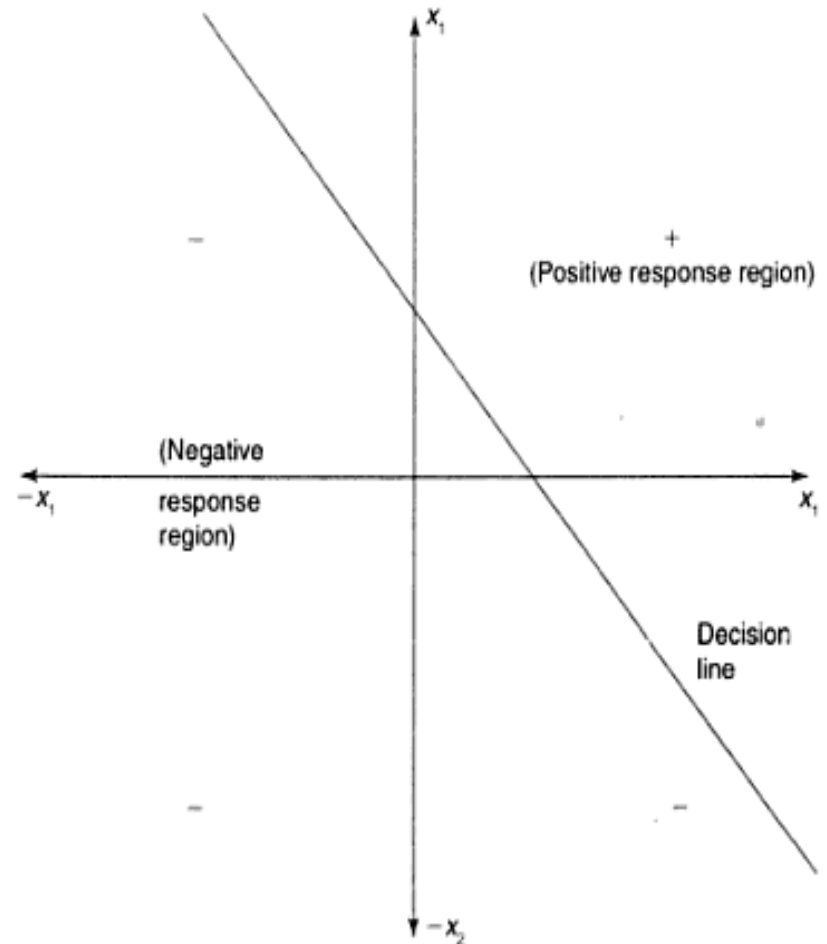
Net requirement for positive response

$$b + x_1 w_1 + x_2 w_2 > 0$$

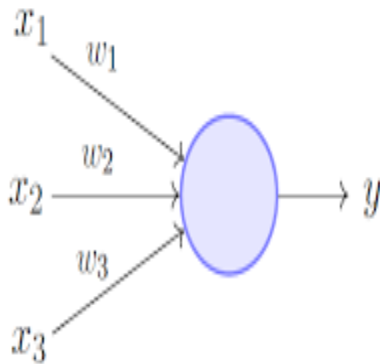
During training process , from training data w_1 , w_2 and b are decided.

Contd..

- Consider a network having positive response in the first quadrant and negative response in all other quadrants with either binary or bipolar data.
- Decision line is drawn separating two regions as shown in Fig.
- Using bipolar data representation, missing data can be distinguished from mistaken data. Hence bipolar data is better than binary data.
- Missing values are represented by 0 and mistakes by reversing the input values from +1 to -1 or vice versa.



Perceptron



Perceptron Model (Minsky-Papert in 1969)

- Frank Rosenblatt, an American psychologist, proposed the **classical perception model** in 1958.
- It is more generalized computational model than the McCulloch-Pitts neuron.
- **Main differences:** Introduction of numerical weights for inputs and a mechanism for learning these weights.
- Inputs are not longer limited to boolean values.
- Refined and carefully analyzed by Minsky and Papert(1969) – their model is referred to as **perceptron** model here.

Perceptron model

$$y=1 \text{ if } \sum_{i=1}^n w_i * x_i \geq \theta$$

$$=0 \text{ if } \sum_{i=1}^n w_i * x_i < \theta$$

Rewriting the above,

$$y=1 \text{ if } \sum_{i=1}^n w_i * x_i - \theta \geq 0$$

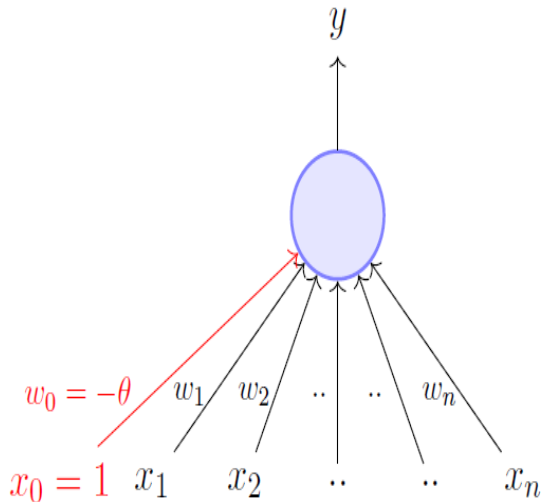
$$=0 \text{ if } \sum_{i=1}^n w_i * x_i - \theta < 0$$

A more accepted convention,

$$y=1 \text{ if } \sum_{i=0}^n w_i * x_i - \theta \geq 0$$

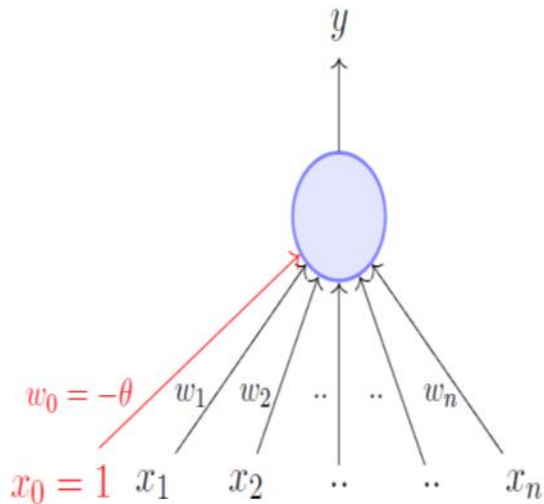
$$=0 \text{ if } \sum_{i=0}^n w_i * x_i - \theta < 0$$

Where $x_0=1$ and $w_0=-\theta$



We will now try to answer the following questions:

- Why are we trying to implement boolean functions?
- Why do we need weights ?
- Why is $w_0 = \theta$ called the bias ?



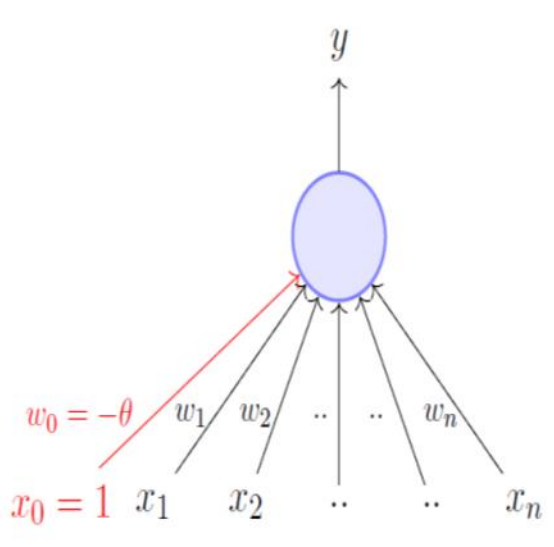
$x_1 = \text{isActorPQR}$

$x_2 = \text{isGenreThriller}$

$x_3 = \text{isDirectorXYZ}$

- Consider the task of predicting whether we would like a movie or not
- Suppose, we base our decision on 3 inputs (binary, for simplicity)
- Based on our past viewing experience (data), we may give a high weight to *isDirectorXYZ* as compared to the other inputs
- Specifically, even if the actor is not *PQR* and the genre is not *thriller* we would still want to cross the threshold by assigning a high weight to *isDirector XYZ*

Weights and Bias



$x_1 = isActorPQR$

$x_2 = isGenreThriller$

$x_3 = isDirectorXYZ$

- w_0 is called the **bias** as it represents the prior (prejudice)
- A movie buff may have a very low threshold and may watch any movie irrespective of the genre, actor, director [$= 0$]
- On the other hand, a selective viewer may only watch *thrillers* starring *PQR* and directed by *XYZ* [$= 3$]
- The weights ($w_1, w_2, \dots, w_n$) and the bias (w_0) will depend on the data (viewer history in this case)

What kind of functions can be implemented using the perceptron? Any difference from McCulloch Pitts neurons?

McCulloch Pitts Neuron

(assuming no inhibitory inputs)

$$y = 1 \quad \text{if } \sum_{i=0}^n x_i \geq 0$$
$$= 0 \quad \text{if } \sum_{i=0}^n x_i < 0$$

Perceptron

$$y = 1 \quad \text{if } \sum_{i=0}^n w_i * x_i \geq 0$$
$$= 0 \quad \text{if } \sum_{i=0}^n w_i * x_i < 0$$

- From the equations it should be clear that even a perceptron separates the input space into two halves
- All inputs which produce a 1 lie on one side and all inputs which produce a 0 lie on the other side
- In other words, a single perceptron can only be used to implement linearly separable functions
- Then what is the difference? The weights (including threshold) can be learned and the inputs can be real valued

Implementing an OR function using perceptron

x_1	x_2	OR	
0	0	0	$w_0 + \sum_{i=1}^2 w_i x_i < 0$
1	0	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
0	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$
1	1	1	$w_0 + \sum_{i=1}^2 w_i x_i \geq 0$

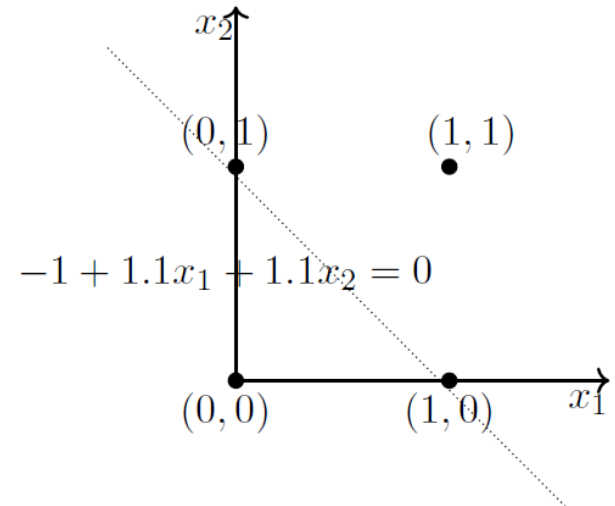
$$w_0 + w_1 \cdot 0 + w_2 \cdot 0 < 0 \implies w_0 < 0$$

$$w_0 + w_1 \cdot 0 + w_2 \cdot 1 \geq 0 \implies w_2 \geq -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 0 \geq 0 \implies w_1 \geq -w_0$$

$$w_0 + w_1 \cdot 1 + w_2 \cdot 1 \geq 0 \implies w_1 + w_2 \geq -w_0$$

- One possible solution to this set of inequalities is $w_0 = -1, w_1 = 1.1, w_2 = 1.1$ (and various other solutions are possible)



- Note that we can come up with a similar set of inequalities and find the value of θ for a McCulloch Pitts neuron also

Review of perceptron

- What about non-boolean (say, real) inputs? **Real valued inputs are allowed in perceptron**
- Do we always need to hand code the threshold? **No, we can learn the threshold**
- Are all inputs equal? What if we want to assign more weight (importance) to some inputs? **A perceptron allows weights to be assigned to inputs**
- What about functions which are not linearly separable ? **Not possible with a single perceptron but can also be handled**